

ATCE

Asservissement numérique d'un oscillateur à quartz

Maxime BELLAUD, Dalina CHABANE,
Khalil FALAH, Paul ROUSSEAU

L3 - S6 - 2025
Semaine du 7 au 11 avril 2025

Table des matières

1	Introduction	2
2	Schéma électrique	2
3	Choix des composants	4
3.1	Choix des composants principaux	4
3.2	Processus de sélection des composants	4
3.3	Liste des composants (BOM)	5
4	Routage du PCB	6
5	Simulation	6
5.1	Analyse des résultats de la simulation AC	7
5.2	Fréquence de résonance de l'oscillateur	7
5.3	Simulation du circuit RC	7
6	Estimation du coût	8
7	Soudure des composants	9
8	Boîtier en impression 3D	9
9	Programmation	10
9.1	Transfert du programme	10
9.2	PWM	11
9.3	ICP	13
9.4	Régulation	14
9.4.1	Réponse indicielle	14
9.4.2	Implémentation du régulateur	15
9.5	Watchdog	16
10	Conclusion	17

11 Bibliographie	18
12 Annexes	19

1 Introduction

L'objectif de ce projet est de réguler la fréquence d'un oscillateur à quartz à l'aide d'un microcontrôleur. Autrement dit, un oscillateur à quartz est sensible à son environnement extérieur, notamment la température, et notre objectif est de faire en sorte que la fréquence de l'oscillateur reste stable malgré ces perturbations.

L'intérêt pour nous est pédagogique car nous réaliserons toutes les étapes pour réaliser ce projet, c'est-à-dire créer le schéma, router le PCB, choisir les composants, souder le circuit, programmer le microcontrôleur, etc. Ce projet nous pousse donc à mettre en œuvre tout ce que nous avons appris pendant nos trois ans de licence, aussi bien en électronique analogique et numérique qu'en automatique.

Ce projet se déroule pendant le dernier semestre de notre licence. Nous réalisons la partie théorique pendant le semestre à l'aide de quelques séances pour nous guider. La réalisation pratique a ensuite lieu pendant une semaine dédiée à ce projet.

Pour réaliser ce projet, nous avons à disposition un PCB routé par nos enseignants ainsi que les composants montés en surface (passifs ou actifs comme le microcontrôleur) pour compléter le circuit. Bien sûr, nous disposons de matériel permettant de les souder. De plus, nous disposons d'oscilloscopes et de multimètres pour réaliser des tests, mais également d'un générateur de signaux afin de simuler un signal 1PPS¹ émit par un satellite. Pour terminer sur la partie matérielle, nous pouvons utiliser une imprimante 3D afin de réaliser une boîte pour le PCB.

Les logiciels que nous utiliserons sont pour la plupart gratuits et libres. Pour réaliser le schéma, la simulation de circuits analogiques et le routage du PCB, nous utilisons KiCAD 9.0. Nous utilisons le langage C pour programmer le microcontrôleur et nous utilisons avr-gcc pour compiler les programmes. De plus, le traitement des informations acquises est réalisé avec MATLAB car c'est un logiciel qui nous est familier. Enfin, nous utilisons FreeCAD pour faire de la modélisation mécanique 3D.

Dans un premier temps, nous présenterons comment nous avons conçu et simulé le circuit, puis routé le PCB. Ensuite se trouvera l'estimation du prix des composants et du PCB en fonction des volumes. Après, nous montrerons comment nous avons assemblé le PCB et imprimé par méthode additive une boîte pour celui-ci. Nous aborderons ensuite la programmation du microcontrôleur avant de passer aux tests et à l'évaluation des performances du système.

2 Schéma électrique

Dans un premier temps, nous réalisons le schéma électrique avec le logiciel KiCAD. Il est représenté sur la figure 1 en annexe. On peut diviser le schéma en plusieurs sous-circuits : le filtre passe-bas, le suiveur, le résonateur et le port USB. Avant de passer à l'explication du schéma, notons que nous avons créé le composant du microcontrôleur sur KiCAD afin de voir clairement toutes ses broches.

1. Signal dont le front montant a lieu exactement toutes les secondes

Alimentation et boutons Nous commençons par alimenter les broches du microcontrôleur avec V_{CC} et GND. De plus, nous ajoutons les boutons RESET et $\overline{HWB}$. Un circuit de pull-up est intégré dans le microcontrôleur pour RESET mais pas pour $\overline{HWB}$ donc nous en réalisons un avec une résistance de $5\text{ k}\Omega$.

Capacités de découplage Pour calculer les valeurs des capacités de découplage, nous utilisons la formule $C = \frac{i \times \Delta t}{\Delta u}$. Nous identifions dans la documentation [1] une consommation d'environ 13 mA ². De plus, à 16 MHz et en choisissant $\Delta u = 0.1\text{ V}$, on obtient $C = 8.125\text{ }\mu\text{F}$, ce que nous arrondissons à $C = 10\text{ }\mu\text{F}$.

Entrées et sorties Le signal 1PPS servant de référence de temps est connecté à la broche 32, ce qui permet d'utiliser la fonction « input capture » (ICP) du timer 3, offrant ainsi les avantages d'un compteur de 16 bits. La sortie PWM est quant à elle connectée à la broche 29 afin de profiter de la résolution de 16 bits du timer 1.

Filtre Le filtre passe-bas en sortie du signal PWM permet d'obtenir une tension continue proportionnelle au rapport de cycle (duty cycle) du PWM. Le suiveur permet ensuite que la suite du circuit n'influence pas cette tension continue qui doit rester la plus précise et stable possible.

Résonateur Le résonateur Y1 est connecté aux broches XTAL1 et XTAL2 du microcontrôleur et on place les capacités de pieds C1 et C2. La varicap (diode dont la capacité change en fonction de sa tension de polarisation) est placée en parallèle d'une capacité de pied, ce qui permet de faire varier la fréquence de résonance de Y1. Cependant, la tension de polarisation de la varicap ne doit pas influencer le résonateur, et le résonateur ne doit pas influencer la polarisation de la varicap. Nous utilisons pour cela un T de polarisation.

Pour le T de polarisation, on détermine les conditions suivantes :

$$\begin{aligned} C_T &\gg C_{\text{osci}} \\ L_T &\gg \frac{1}{4\pi f^2 C_T^2} \end{aligned}$$

C_{osci} sont les capacités de pieds de l'oscillateur, qui sont de l'ordre de 20 pF , donc nous choisissons $C_T = 1\text{ }\mu\text{F}$. On en déduit alors $L_T = 10\text{ }\mu\text{H}$.

À très basse fréquence, c'est-à-dire en continu, on a $X_{LT} \rightarrow 0$ et $X_{CT} \rightarrow \infty$, ce qui confirme que la tension continue n'influence pas le résonateur. À 16 MHz , on obtient $X_{LT} \approx 1\text{ k}\Omega$ et $X_{CT} \approx 10\text{ m}\Omega$, ce qui indique que l'oscillateur « voit » bien la varicap mais n'influence pas sa tension de polarisation continue car $X_{CT} \gg X_{LT}$.

Port USB Le port USB est simplement connecté comme indiqué dans la documentation du microcontrôleur [1]. La broche ID est laissée déconnectée car cela a peu d'importance pour nous. Enfin, la résistance de 0Ω nous permet de tester que l'alimentation de l'USB est correcte avant de connecter le reste du circuit lors de l'assemblage.

2. Figure 30-4

Points de test Enfin, nous ajoutons des points de tests afin de pouvoir vérifier le fonctionnement du système lors de la conception du programme du microcontrôleur. Nous choisissons cependant de ne pas en mettre dans le circuit du résonateur afin de ne pas le perturber.

3 Choix des composants

3.1 Choix des composants principaux

Microcontrôleur : L'Atmega32U4 a été imposé pour ce projet, notamment parce que nous l'avons déjà beaucoup utilisé, ce qui facilitera le développement du programme. De plus, ses timers 16 bits pouvant générer un signal PWM et utiliser la fonction « input capture » sont intéressants. Son boîtier TQFP-44 (SMD) est compatible avec un montage en surface et une soudure manuelle, et il est disponible chez Farnell (par exemple, référence 1748525, prix unitaire d'environ 5,250 €).

Connecteur USB : Un connecteur USB mini-B horizontal (ref `USB_OTG_SMD_xxx`) a été choisi pour son faible encombrement et sa robustesse mécanique. De plus, c'est le connecteur standard des cartes électroniques que l'on utilise, ce qui signifie que nous n'avons pas besoin d'acheter de câble. Ce modèle SMD facilite le soudage et l'alignement sur la carte, et est largement disponible (par exemple, Molex sur Farnell, référence 3500674, environ 1,00 €).

Résonateur à quartz : Un quartz SMD Abracon ABM3 à 16 MHz (5,0×3,2 mm, empreinte `Crystal_SMD_Abracon_ABM3`) a été retenu pour sa précision, son faible encombrement, et sa compatibilité avec l'asservissement GPS. Il est également disponible chez Farnell à 0,258 €.

Varicap : La diode varicap BB833 a été remplacée par la BB857, plus disponible, avec une plage de capacité (5 pF à 10 pF) adaptée à la correction de la fréquence, une tension inverse compatible (jusqu'à 5 V), et un boîtier SMD 0805 (2012 Metric) facile à souder. Elle est disponible chez Farnell (référence 2725709, environ 0,172 €).

Boîtiers des composants passifs : Les boîtiers SMD 0805 ont été privilégiés pour les résistances et condensateurs (par exemple, R5 à 5 k Ω , C5 à 10 μ F), offrant une taille réduite et une densité élevée sur le PCB, tout en restant maniables pour une soudure manuelle. Le format 1206 a été utilisé pour des composants nécessitant une dissipation de puissance minimale (par exemple, R1 à 0 Ω). Ces formats standards garantissent une disponibilité chez les fournisseurs comme Farnell.

3.2 Processus de sélection des composants

Pour chaque composant, nous avons suivi une méthodologie spécifique afin de garantir leur compatibilité avec notre design et nos contraintes de fabrication :

Fixation de la taille des boîtiers et des empreintes : Nous avons défini des boîtiers SMD (principalement 0805, avec quelques 1206 pour des besoins spécifiques) pour tous les composants passifs, car ils sont adaptés à un montage en surface et à une soudure manuelle. Les empreintes correspondantes ont été choisies dans la bibliothèque KiCad (par exemple, `Resistor_SMD:R_0805_2012Metric` pour une résistance 0805, ou `Capacitor_SMD:C_0805_2012Metric` pour un condensateur 0805). Pour les composants plus complexes comme le quartz ou la varicap, des empreintes spécifiques ont été sélectionnées (par exemple, `Crystal_SMD_Abracon_ABM3` pour le quartz).

Sélection des composants sur Farnell : Les composants ont été choisis sur Farnell en fonction de plusieurs critères :

- **Valeur électrique :** Par exemple, un condensateur de 10 μF pour le découplage ou une résistance de 5 k Ω pour le filtre RC.
- **Capacité et tolérance :** Pour les condensateurs, une tolérance de $\pm 5\%$ a été privilégiée pour les applications de découplage et de circuits de filtrage. Pour les résistances, une tolérance de $\pm 1\%$ a été choisie pour plus de précision.
- **Dimensions et boîtier :** Les dimensions ont été vérifiées pour correspondre aux empreintes KiCad (par exemple, un boîtier 0805 mesure 2,0×1,2 mm).
- **Prix unitaire et disponibilité :** Les prix unitaires ont été comparés pour optimiser les coûts. Par exemple, pour un condensateur de 1 μF en boîtier 0805, nous avons comparé plusieurs références sur Farnell. La référence 4166923 a été retenue, avec un prix unitaire de 0,121 €, une tolérance de $\pm 5\%$, et une disponibilité suffisante pour notre projet.

Exemple de sélection : Prenons l'exemple du condensateur C5 (10 μF) utilisé dans le filtre RC avec R5. Nous avons d'abord fixé un boîtier 0805 pour ce condensateur, en raison de sa taille compacte et de sa compatibilité avec une soudure manuelle. Dans KiCad, l'empreinte `Capacitor_SMD:C_0805_2012Metric` a été assignée. Sur Farnell, nous avons recherché des condensateurs SMD de 10 μF en boîtier 0805. Parmi les options, la référence 2470435 a été choisie : elle offre une tolérance de $\pm 10\%$, une tension nominale de 16 V (supérieure à notre alimentation de 5 V), un prix unitaire de 1,118 €, et une disponibilité immédiate (plus de 10 000 unités en stock). Une alternative, la référence 2581105 à 1.230 €, était légèrement plus chère pour des caractéristiques similaires, donc nous avons privilégié la première option.

3.3 Liste des composants (BOM)

Après avoir sélectionné tous les composants, nous avons généré la nomenclature (BOM) dans KiCad pour regrouper et organiser les informations, qu'on peut retrouver en annexe Figure 2 ou à télécharger en [2]. Voici les étapes suivies :

- **Ajout des prix à la BOM :** Pour chaque composant, le prix unitaire (obtenu sur Farnell) a été ajouté manuellement dans les champs personnalisés de KiCad (via Eeschema, en éditant les propriétés des composants). Par exemple, pour le condensateur 4166923, le champ "Prix" a été rempli avec "0,121 €".
- **Groupement par valeur :** Dans l'outil de génération de BOM de KiCad (accessible via "Fabriquer > Liste des composants"), nous avons configuré le script pour regrouper les composants par valeur. Par exemple, tous les condensateurs de 1 μF

en boîtier 0805 ont été regroupés sous une seule ligne, avec une quantité totale (par exemple, 3 unités pour C5, C10, etc.).

- **Exportation de la BOM** : La BOM a été exportée au format CSV pour faciliter son utilisation lors de la commande des composants. Le fichier inclut les colonnes suivantes : référence, valeur, empreinte, quantité, prix unitaire, et lien vers la fiche technique sur Farnell. Cela permet une commande rapide et une vérification finale avant achat.
- **La disponibilité du composant** : Sur Farnell, il faut vérifier la disponibilité de notre besoin ainsi que la possibilité d'en commander en quantité suffisante ont également été prises en compte.

4 Routage du PCB

Le routage commence par la récupération des empreintes, suivie du placement des composants selon la logique suivante : l'Atmega est placé en premier, au centre de la carte. Le quartz est positionné au plus proche pour ne pas perturber la stabilité de l'oscillateur. Les capacités de découplage sont rapprochées des broches d'alimentation, tandis que le connecteur USB est disposé de façon à optimiser le routage des paires différentielles.

Une fois que ces éléments importants sont positionnés, on dessine le contour du PCB avec la couche (Edge.Cuts). Les autres composants sont ensuite répartis de manière à limiter la longueur des pistes et à éviter les croisements.

Le routage suit le même ordre : le groupe avec le quartz, les capacités de découplage, USB, puis le reste du circuit. Les empreintes sont déplacées pour faciliter le routage de certaines pistes. Le plan 5V est relié directement sur la couche du haut, tandis que le plan de masse, situé sur la deuxième couche, est connecté à l'aide de vias traversants.

On effectue le contrôle de conception (DRC) pour corriger toutes les erreurs ou avertissements, notamment ceux liés à la sérigraphie (F.SilkScreen) que l'on repositionne.

Cette méthode est celle pour obtenir un PCB double couche (voir figure 3). Puis pour la version monocouche le principal défi consiste à superposer les plans 5V et de masse sur une même couche, ce qui impose de déplacer certaines empreintes (voir figure 4).

Les paramètres du routage sont faits pour respecter la classe 4, mais l'empreinte du connecteur BNC impose des vias traversants de classe 6. En dernier, on ajoute des perçages aux coins du PCB pour permettre sa fixation dans un boîtier.

5 Simulation

Le schéma en figure 5 réalisé pour la simulation, représente un oscillateur à quartz avec une fréquence ajustable par une varicap, divisé en trois sections principales. La première section génère une tension de commande via une source PWM (B1, fréquence 10 kHz, rapport cyclique 0,5) filtrée par un RC passe-bas ($R5 = 5 \text{ k}\Omega$, $C5 = 10 \text{ }\mu\text{F}$), produisant une tension continue au nœud `PWM_filtered`. Cette tension est appliquée à un suiveur de tension (E1, modèle idéal, gain 1), dont la sortie (`Follower_output`) polarise la varicap. La deuxième section est l'oscillateur à quartz, modélisé par un circuit Butterworth-van Dyke [3] ($L2 = 5 \text{ mH}$, $C6 = 19,78929368 \text{ fF}$, $R1 = 50 \text{ }\Omega$, $C7 = 7 \text{ pF}$), avec des condensateurs de charge C9 (15 pF) et C8 (10 pF) pour satisfaire les conditions de Barkhausen. Une source AC (B2, 1 V) est ajoutée pour l'analyse en fréquence. La

troisième section ajuste la fréquence via une varicap ($C11 = 2 \text{ pF}$ à 10 pF), connectée entre XTAL2 et Follower_output, permettant un réglage fin de la fréquence de résonance.

5.1 Analyse des résultats de la simulation AC

L'analyse AC, réalisée sur une plage de fréquences de $15,9 \text{ MHz}$ à $16,1 \text{ MHz}$, a permis d'identifier la fréquence de résonance du quartz à $16,017200 \text{ MHz}$ pour un rapport cyclique de $0,5$ (tension de commande moyenne de $2,5 \text{ V}$) et une capacité de la varicap ($C11$) fixée à 2 pF . En faisant varier la capacité de la varicap de 2 pF à 10 pF , nous avons observé un décalage de la fréquence : à 6 pF , la fréquence est de $16,012000 \text{ MHz}$, et à 10 pF , elle descend à $16,009200 \text{ MHz}$ comme on peut le voir en figure 6. Cette variation de 8 pF (de 2 pF à 10 pF) entraîne un décalage total de 8 kHz (de $16,017200 \text{ MHz}$ à $16,009200 \text{ MHz}$), soit environ 1 kHz par pF . Ces résultats confirment que la varicap permet un ajustement fin de la fréquence, avec une plage de réglage suffisante pour compenser les dérives de l'oscillateur (de 30 à 50 ppm , soit 480 à 800 Hz pour un quartz à 16 MHz) dans le cadre de l'asservissement GPS.

5.2 Fréquence de résonance de l'oscillateur

L'analyse a été réalisée pour déterminer la fréquence de résonance de l'oscillateur à quartz. Le gain maximal, mesuré à 15 dB sur le nœud $V(\text{XTAL2})$, est atteint pour une fréquence de $16,009200 \text{ MHz}$, ce qui correspond à la fréquence de résonance. Cette fréquence est associée à une capacité de la varicap ($C11$) de 10 pF , comme confirmé par les simulations précédentes (2 pF à $16,017200 \text{ MHz}$, 6 pF à $16,012000 \text{ MHz}$, et 10 pF à $16,009200 \text{ MHz}$).

En comparaison avec la théorie, la fréquence de résonance attendue, calculée à partir du modèle Butterworth-van Dyke en annexe [3] ($L2 = 5 \text{ mH}$, $C6 = 19,78929368 \text{ fF}$), est de $f_0 = \frac{1}{2\pi\sqrt{L2 \cdot C6}} \approx 16,009190 \text{ MHz}$. L'écart entre la valeur mesurée ($16,009200 \text{ MHz}$) et la valeur théorique ($16,009190 \text{ MHz}$) est de $0,6 \text{ ppm}$, ce qui est négligeable et probablement dû aux capacités parasites et à la précision numérique de ngspice. Ces résultats valident la conception de l'oscillateur pour une utilisation dans l'asservissement GPS.

5.3 Simulation du circuit RC

Pour ce qui est de la simulation du circuit RC et afin de déterminer les meilleurs choix pour la résistance et la capacité, on procédera de la façon suivante :

Exemple avec $R = 5,1 \text{ k}\Omega$ et $C = 1 \mu\text{F}$. On simulera toujours avec une fréquence de 7812 Hz (voir le calcul de cette fréquence à la partie 9.2) et un rapport cyclique de 50% . Grâce à notre simulation, on relève la constante de temps τ . Pour ce faire, on utilisera la méthode des 95% de la valeur finale, puis par projection à partir de $t = 0$ on observe et on relève, dans le cas de la figure en annexe (voir figure 8), $\tau = 16,4 \text{ ms}$.

On peut comparer ce résultat avec le calcul théorique. En effet, le produit RC donne la constante de temps :

$$\tau_{th} = R \times C = 5,1 \times 10^3 \times 1 \times 10^{-6} = 5,1 \text{ ms}.$$

Or, dans un circuit RC, le temps pour atteindre environ 95% de la valeur finale est approximativement donné par :

$$T_{95\%} \approx 3 \tau_{th} = 3 \times 5,1 \text{ ms} = 15,3 \text{ ms},$$

ce qui est proche de la valeur mesurée.

Une fois τ relevé, on relève également la variation de tension lorsque la réponse se stabilise. Comme on peut le voir dans la figure en annexe (voir figure 9), il suffit d'utiliser les curseurs pour relever environ 24 mV.

On appliquera la même procédure pour d'autres valeurs de R et C , comme présenté dans le tableau ci-dessous :

R	C	$T_{95\%}$	ΔV
5.1 k Ω	0.1 μ F	2.27 ms	286 mV
5.1 k Ω	1 μ F	16.4 ms	24 mV
5.1 k Ω	10 μ F	150 ms	3.09 mV
3 k Ω	0.1 μ F	2.35 ms	312 mV
3 k Ω	1 μ F	16.4 ms	30.8 mV
3 k Ω	10 μ F	153 ms	3.09 mV

On remarque alors que, selon nos observations, les meilleurs choix sont $R = 5.1 \text{ k}\Omega$ et $C = 1 \mu\text{F}$, ce qui représente le meilleur compromis entre temps de réponse et stabilité de la tension.

6 Estimation du coût

D'après le site Eurocircuits (voir Figure 7), pour un prototype de PCB à 2 couches, le prix est de 40,99 € (net) avec un délai de 3 jours. De plus, l'éditeur de classification indique que notre PCB est conçu en classe 6, ce qui correspond à des largeurs de pistes et des espacements plus exigeants.

TABLE 1 – Prix unitaire (valeur en haut de la case) et total (valeur en bas de la case) en € pour différentes quantités et délais

Délai (jours)	1 PCB	5 PCB	10 PCB	15 PCB	20 PCB
2 jours	100,53	28,15	20,00	16,00	10,49
	100,53	140,75	200,00	240,00	209,80
3 jours	40,99	12,16	13,82	10,70	8,35
	40,99	60,80	138,20	160,50	167,00
4 jours	40,99	12,16	10,57	8,10	6,22
	40,99	60,80	105,70	121,50	124,40
5 jours	40,99	12,16	7,32	5,50	4,43
	40,99	60,80	73,20	82,50	88,60
6 jours	40,99	12,16	7,32	5,50	4,43
	40,99	60,80	73,20	82,50	88,60
7 jours	40,99	12,16	7,32	5,50	4,43
	40,99	60,80	73,20	82,50	88,60

La matrice de prix montre clairement comment le coût varie en fonction du délai de fabrication et de la quantité commandée. Chaque case présente le prix unitaire ainsi que

le montant total pour la commande. Par exemple, pour une commande d'un seul PCB, le tarif passe de 100,53 € avec un délai de 2 jours à 40,99 € dès que le délai atteint 3 jours, où il se stabilise. Pour une commande de 5 PCB, le même phénomène se produit avec une forte réduction du prix unitaire entre 2 et 3 jours, puis une stabilité dès 3 jours. En revanche, pour des commandes plus importantes comme 10, 15 ou 20 PCB, le prix unitaire continue de baisser jusqu'à atteindre un délai de fabrication de 5 jours, après quoi il se fixe. Cela montre qu'en choisissant un délai de fabrication légèrement plus long, on peut réduire significativement le coût unitaire, en particulier pour des commandes groupées. Ainsi, commander 5 PCB par exemple s'avère être une option intéressante, permettant de bénéficier d'un tarif avantageux tout en obtenant suffisamment d'exemplaires pour divers besoins (plusieurs test ou autre).

7 Soudure des composants

Au début de la semaine de projet nous avons un PCB nu, l'objectif est de souder les composants afin d'obtenir une carte fonctionnelle. C'est un moyen pour nous de découvrir la soudure de composants montés en surface, notre expérience était limitée aux composants traversants.

Pour faciliter l'implantation des composants sur le PCB, le plug-in « Interactive HTML BOM » dans KiCad nous permet de localiser précisément chaque composant sur la carte.

Dans un premier temps, nous définissons l'ordre des composants à souder. Le connecteur USB est placé en premier pour vérifier la présence du 5 V et de la masse, suivi des composants les plus petits (diodes, résistances, condensateurs), puis de l'Atmega, et enfin des composants traversants (boutons, connecteur BNC). Cet ordre nous permet d'avoir le plus d'espace de travail maximal pour les composants les plus délicats à souder.

La soudure de l'USB s'avère complexe et nécessite plusieurs essais. Un court-circuit se produit lorsqu'une bille d'étain retombe sur les broches du connecteur, les reliant entre elles. On essaye une première fois de le dessouder à la tresse, sans succès. Nous avons donc demandé de l'aide au technicien qui a retiré le connecteur avec un pistolet à air chaud. Le vernis à donc fondu dans cette partie du PCB. Après nettoyage de l'étain sur l'USB et la suppression d'une plaque métallique à l'arrière du connecteur, qui était la source du court-circuit. On a pu ressouder le port USB correctement (voir la figure 10). La suite de la soudure se déroule sans problème.

Pour conclure, l'utilisation de la loupe binoculaire nous a demandé un temps d'adaptation, chaque membre du groupe a eu l'occasion de participer à l'assemblage, à l'exception d'un membre qui préfère ne pas intervenir. Tout au long de l'assemblage du PCB, nous avons vérifié au multimètre pour détecter d'éventuels courts-circuits et contrôler la présence de la tension d'alimentation. La soudure est achevée dès le mardi midi. Nous avons pu vérifier le fonctionnement de la carte, ce qui nous a permis d'aborder les étapes suivantes du projet rapidement.

8 Boîtier en impression 3D

Pour compléter le projet, nous décidons de concevoir un boîtier pour accueillir le PCB. Les contraintes principales sont de respecter les dimensions de la carte, de permettre sa fixation tout en assurant de pouvoir le mettre et l'enlever facilement, et de conserver l'accessibilité aux deux boutons, au port USB et au connecteur BNC.

Le modèle 3D du PCB est exporté depuis KiCad, puis importé dans FreeCAD qui est le logiciel que nous avons utilisé pour modéliser. La boîte est pensée pour être imprimée en 3D et se compose de trois parties distinctes : le corps principal, un couvercle clipsable et des rallonges de boutons. Pour l'impression 3D, on utilise une PRUSA MK3+ qui nous est mis à disposition en salle CMI.

Le corps de la boîte est rectangulaire aux dimensions du PCB. Quatre supports sont modélisés au fond pour maintenir la carte en place. Des ouvertures spécifiques sont prévues pour permettre l'accès au connecteur BNC, aux boutons et au port USB. Des aérations sont ajoutés pour limiter la chaleur à l'intérieur. Au-dessus des boutons, on ajoute des marquages pour distinguer les boutons « Reset » et « HWB ».

Une première version est imprimée mais a plusieurs défauts, notamment l'encombrement du connecteur BNC qui empêche d'insérer correctement le PCB, et la fixation n'est pas satisfaisante. Une seconde version corrige ces problèmes en modifiant le type de fixation tout en améliorant certains détails.

Le couvercle est conçu aux mêmes dimensions que la boîte. Il comporte des chanfreins sur ces contours qui permettent de le fermer correctement sur la boîte. Les boutons, étant légèrement en retrait dans la boîte ils sont donc difficilement accessibles. Pour y remédier, on modélise des rallonges qui viennent se fixer directement sur la surface ronde des boutons.

L'impression s'est bien passé, cependant nous avons eu des difficultés lors du retrait des supports. L'un d'eux nécessite l'usage d'une fraiseuse puis d'une lame chauffée pour être retiré proprement. Malgré cela, le boîtier final (voir figure 17) est fonctionnel et conforme au cahier des charges que nous nous sommes fixés.

9 Programmation

Une fois le circuit physique réalisé, il ne reste qu'à programmer le microcontrôleur afin que le système réalise ce qu'on attend de lui. Nous disposons d'un circuit quasiment complet afin qu'une partie du groupe puisse commencer la programmation pendant que l'autre partie assemble le PCB.

Tous les codes sont disponibles sur le dépôt Github : <https://github.com/Pauloux/Projet-ATCE>.

9.1 Transfert du programme

Dans un premier temps, nous souhaitons démontrer notre capacité à programmer le microcontrôleur. En effet, il est vierge et nous ne pouvons pas utiliser avrdude comme nous le faisons pendant le cours de microcontrôleur cette année.

Code Le code (fichier Hello_world.c) est simplement composé de la déclaration d'une LED en sortie (PORTF1), puis de sa commutation toutes les 0.5s dans une boucle infinie. À noter que nous utilisons la bibliothèque LUFA car ses fonctions d'initialisation permettent d'indiquer au microcontrôleur d'utiliser l'oscillateur externe de 16 MHz et non celui de 1 MHz intégré.

Nous ajoutons également la possibilité de communiquer par USB pour tester la bibliothèque LUFA, mais nous commentons cette partie du code le temps de réaliser les mesures de temps sur la LED qui clignote. En effet, la communication USB prend du temps et fausse les durées, notamment lorsque le port USB n'est pas sondé par l'ordinateur.

Makefile Nous réutilisons un Makefile que nous avons utilisé cette année en cours et nous l'adaptions à notre programme. De plus, c'est ce Makefile qui sera utilisé pour tous les autres programmes réalisés dans ce projet.

Le plus gros changement se trouve dans le transfert du programme. Nous utilisons « dfu-programmer » (qu'il faut installer sur nos machines) pour cela. Nous réalisons donc le Makefile qui permet d'exécuter les commandes suivantes pour transférer le programme, comme nous l'avait indiqué M. FRIEDT lors des cours de préparation du projet :

- dfu-programmer atmega32u4 erase
- dfu-programmer atmega32u4 Hello_world.hex
- dfu-programmer atmega32u4 reset

Test Nous testons le programme avec son Makefile et on observe que la LED correspondant au port F1 clignote bien au rythme de 1 Hz. De plus, le message « Hello world » s'affiche bien dans la console lorsque nous utilisons le logiciel minicom.

Nous remercions M. FRIEDT qui nous a indiqué la nécessité d'inclure les fonctions d'initialisation de la bibliothèque LUFA afin que le microcontrôleur utilise l'horloge de 16 MHz. En effet, lors de nos premiers tests, les délais étaient incorrects car nous avons indiqué que la fréquence est de 16 MHz dans les programmes (notamment pour la fonction `_delay_ms`) alors qu'elle était en réalité de 1 MHz.

Maintenant que nous sommes capables de programmer le microcontrôleur, nous pouvons passer à la programmation de fonctionnalités utiles pour le projet.

9.2 PWM

Le microcontrôleur ATMEGA32U4 ne comporte pas directement de convertisseur numérique-analogique (DAC). Ainsi, afin de pouvoir générer une tension continue dont on peut contrôler l'amplitude, nous utilisons un signal à rapport cyclique variable (PWM) suivi d'un filtre passe-bas afin d'extraire la valeur moyenne de ce signal. Concrètement, nous voulons générer un signal créneau en pouvant modifier le temps passé à l'état haut par rapport au temps passé à l'état bas, ce qui permet de modifier la valeur moyenne.

Ce programme s'appelle « PWM.c » sur le dépôt Github.

Identification du timer Sur le schéma KiCAD fourni par nos enseignants, nous observons que le filtre passe-bas est connecté au port D7 du microcontrôleur, ce qui correspond à une sortie du timer 4 (OC4D). Nous devons donc utiliser le timer 4, qui a la spécificité d'être sur 10 bits.

Fonctionnement et registres Un schéma représentant le fonctionnement du timer en mode PWM est représenté sur la figure 11 en annexe.

Deux registres sont importants pour contrôler le signal : OCR4C permet de modifier la fréquence du signal et OCR4D permet de changer le rapport cyclique.

OCR4C permet de changer la fréquence car il permet de contrôler jusqu'à quelle valeur le compteur va compter avant de redescendre. Ainsi, si OCR4C diminue, alors le compteur atteindra plus vite sa valeur maximale et fera des allers-retours plus vite. S'il fait des allers-retours plus vite, alors la fréquence du signal de sortie augmente.

On observe sur le schéma que le port D7 change d'état lorsque la valeur du timer 4 est égale à la valeur de OCR4D. Plus précisément, PORTD7 passe à l'état bas lorsque

cet événement se produit et que le compteur compte vers le haut, et PORTD7 passe à l'état haut lorsque cet événement se produit et que le compteur compte vers le bas. Ainsi, si OCR4D est proche de 0, alors le port D7 sera la plupart du temps à l'état bas. Au contraire, si OCR4D est proche de OCR4C, alors le port D7 sera la plupart du temps à l'état haut.

En jouant sur les valeurs de OCR4D, on peut alors modifier le rapport cyclique du signal sans modifier sa fréquence (contrôlée par OCR4C). À noter que si OCR4D = 0, alors le signal est toujours à l'état bas, et si OCR4D = OCR4C, alors le signal est toujours à l'état haut.

Utilisation en mode 10 bits La spécificité du timer 4 est qu'il utilise 10 bits tandis que le microcontrôleur utilise des registres de 8 bits. Pour stocker des valeurs sur 10 bits, on utilise le registre TC4H. Pour enregistrer une valeur de 10 bits dans un registre (OCR4C pour l'exemple), il suffit alors d'assigner les 2 bits de poids forts dans le registre TC4H puis d'assigner les 8 bits de poids faibles dans le registre dont on veut assigner la valeur de 10 bits (ici OCR4C). Lorsque ce dernier registre est assigné (ici OCR4C), le microcontrôleur place les bits de poids forts stockés dans TC4H et les sauvegarde dans le registre voulu (ici OCR4C). L'ordre d'assignation des registres est donc important et il faut toujours commencer par assigner TC4H.

C'est un fonctionnement simple et astucieux même s'il peut être contre-intuitif, notamment car nous ne comprenons pas exactement où sont stockés les bits de poids forts une fois l'assignation réalisée.

Valeurs des registres Nous souhaitons pouvoir être le plus précis possible au niveau du rapport cyclique. De par le fonctionnement du système, OCR4D est forcément inférieur à OCR4C. Ainsi, pour avoir le plus de valeurs possibles pour OCR4D, alors OCR4C doit être le plus élevé possible, c'est-à-dire 1023.

La fréquence du signal PWM est donné par :

$$f_{\text{PWM}} = \frac{16 \text{ MHz}}{N \times 2 \times (1 + \text{OCR4C})}$$

En utilisant le plus petit prescaler, c'est-à-dire 1, cela nous donne une fréquence du signal de $f_{\text{PWM}} = 7812.5 \text{ Hz}$.

Fonctions Pour contrôler le plus simplement possible le signal PWM, nous créons deux fonctions. La première sert à initialiser la fonction PWM tandis que la deuxième permet de changer le rapport cyclique du signal. Cette deuxième fonction prend simplement un nombre entre 0 et 1023 et l'assigne au registre OCR4D.

Test Afin de tester la fonction qui permet de modifier le rapport cyclique, nous réalisons un algorithme qui le fait varier de 0 à 100 % puis recommence de 0. De plus, le rapport cyclique reste à 0 % et 100 % quelques secondes afin de vérifier que les valeurs extrêmes se comportent comme attendu.

Nous visualisons alors le signal en sortie du port D7 à l'oscilloscope à l'aide du point de test sur le PCB. Nous observons que le signal a bien une fréquence de 7812.5 Hz et que le rapport cyclique varie de 0 % à 100 %. De plus, on a bien une tension continue de 0 V lorsque le rapport cyclique est de 0 % et une tension continue de 5 V lorsque le rapport cyclique est de 100 %.

9.3 ICP

Nous souhaitons maintenant pouvoir mesurer la fréquence de l'oscillateur. Pour cela, nous utilisons la fonction « input capture » (ICP) d'un timer.

Ce programme s'appelle « ICP.c » sur le dépôt Github.

Identification du timer Le signal 1 PPS (1 impulsion par seconde) est connecté à la broche correspondant au port D4, c'est-à-dire à l'entrée ICP du timer 1.

Fonctionnement Un schéma représentant le fonctionnement du timer en mode ICP est représenté sur la figure 12 en annexe.

Le timer 1 compte continuellement vers le haut et, lorsqu'il atteint sa valeur maximale (2^{16} car il est sur 16 bits), il recommence à compter à 0. À l'aide d'une interruption, nous pouvons détecter lorsque cela se produit (overflow) et ainsi compter le nombre de cycles réalisés.

Lorsqu'un front montant intervient sur le signal 1PPS, une interruption se déclenche et nous permet de lire l'état du compteur (ICR1). Nous prenons ensuite soin de sauvegarder le nombre de cycles réalisés ainsi que l'état du compteur lorsque cela survient, puis nous remettons à 0 le compteur ainsi que la variable correspondant au nombre de cycle réalisé. Cela permet au compteur de toujours commencer dans les mêmes conditions et donc simplifier le traitement de l'information ensuite. Enfin, nous activons un drapeau qui permettra à la boucle principale infinie de traiter l'information lorsqu'elle en a le temps et ainsi ne pas ralentir l'exécution de l'interruption.

En sélectionnant un prescaler de 1, cela signifie que le timer 1 compte à 16 MHz et doit donc normalement avoir compté jusqu'à 16×10^6 lorsque le signal 1 PPS s'active. Si l'oscillateur est en réalité plus rapide, alors le compteur aura le temps de compter plus de fois pendant 1 s, et le même raisonnement s'applique si l'oscillateur est plus lent. Ainsi, on peut écrire la formule :

$$f_{\text{mesuree}} = 2^{16} \times \text{nbr cycles} + \text{ICR1}$$

Valeurs attendues À 16 MHz, on a 244 cycles + 9216 (ICR1). De plus, en considérant dans le pire cas que l'oscillateur a une précision de 50 ppm, cela fait un écart de fréquence de $16 \times 10^6 \times 50 \times 10^{-6} = 800$ Hz, ce qui est largement inférieur à 9216 Hz. En effet, 9216 Hz correspond à un écart de 576 ppm dans notre cas, ce qui n'arrivera normalement pas. On peut donc considérer que le nombre de cycle n'est pas important car il sera normalement toujours égal à 244, et seule la valeur lue dans ICR1 lorsque l'interruption se déclenche est intéressante pour nous.

Fonctions et interruptions Comme expliqué précédemment, nous utilisons deux interruptions : une pour gérer le débordement (overflow) du timer, tandis que l'autre intervient lorsque le front montant du signal 1 PPS est détecté. À noter qu'il faut utiliser des variables préfixées de « volatile » afin qu'elles ne soient pas optimisées et que l'on puisse modifier leur valeur à l'intérieur des interruptions. De plus, nous créons une fonction permettant de simplement initialiser le timer 1.

Code de test Pour tester si le code fonctionne correctement, nous envoyons simplement par USB les mesures prises (nombre de cycles et ICR1) toutes les secondes. Nous ne faisons

évidemment pas cela dans l'interruption mais dans la boucle principale infinie. Ce code ne s'exécute que si le drapeau indiquant qu'une mesure a été faite est actif, ce qui assure que chaque mesure n'est envoyée qu'une fois et toute les secondes.

Test Afin de simuler le signal 1 PPS, nous utilisons un générateur de signaux réglé à 1 Hz dont l'amplitude varie entre 0 et 3.3 V. De plus, nous réglons son rapport cyclique au minimum, c'est-à-dire 20 % pour se rapprocher au maximum d'un véritable signal 1 PPS, même si cela n'a normalement pas d'importance.

Lorsque l'on exécute le programme, les valeurs observées sont cohérentes, c'est-à-dire que le nombre de cycle est toujours égal à 244 et la valeur de ICR1 est autour de 8500, ce qui correspond alors à une fréquence de 15 999 284 Hz. De plus, nous faisons chauffer le résonateur à l'aide d'un fer à souder afin de voir l'impact de la température sur la fréquence de l'oscillateur. Nous enregistrons les valeurs dans un fichier à l'aide de la commande « minicom -D /dev/ttyACM0 -C frequence.dat » et nous traçons l'évolution de la fréquence mesurée en fonction du temps. Pour cela, nous utilisons un script MATLAB qui trace simplement l'évolution de ICR1. Nous obtenons alors le graphique de la figure 13 en annexe.

On voit clairement le moment où le fer à souder est posé sur le résonateur car la fréquence diminue d'abord légèrement puis augmente drastiquement. Autrement dit, « avant la montagne se trouve le fossé ». L'écart de fréquence observé est d'environ 400 Hz, ce qui est l'ordre de grandeur que l'on attendait.

Nous considérons que cette partie est fonctionnelle et que l'on peut mesurer précisément la fréquence de l'oscillateur. De plus, nous sommes capables de tracer l'évolution de la fréquence et de vérifier que la température l'influence bel et bien.

9.4 Régulation

Maintenant que nous pouvons contrôler le procédé (la fréquence de résonance du résonateur) avec le programme PWM et le mesurer avec le programme ICP, nous pouvons réguler le système. L'objectif est de mesurer la fréquence de l'oscillateur et de polariser la varicap correctement afin que la fréquence reste stable et devienne insensible aux perturbations comme la température par exemple. Ce programme s'appelle « Asservissement_PL.c » sur le dépôt Github.

Nous utilisons pour cela un régulateur PID et en calculant les coefficients avec la méthode de Takahashi [6]. Nous avons pour cela besoin d'informations tirées de la réponse indicielle du système.

En traçant l'évolution de la fréquence mesurée en fonction du rapport cyclique du signal PWM, et donc de la tension de polarisation de la varicap (figure 14 en annexe), on observe que la caractéristique n'est pas linéaire. Nous travaillerons donc au milieu de la courbe pour la régulation.

9.4.1 Réponse indicielle

Pour tracer la réponse indicielle, nous réalisons un programme qui utilise les fonctions précédemment créées afin de générer un signal PWM et de mesurer la fréquence de l'oscillateur. L'algorithme dans la boucle principale infinie permet de générer un signal créneau pour la valeur du rapport cyclique du PWM. Concrètement, le rapport cyclique est de 462 sur 1023 pendant 5 s, puis il passe à 562 sur 1023 pendant 5 s avant que le cycle ne recommence. Cela permet de rester dans une gamme de ± 50 sur 1023 de rapport cyclique.

Nous enregistrons les valeurs d'un certain nombre (24 mesures retenues) de réponse indicielle et nous réalisons une moyenne (15). À partir de ce graphique, nous déterminons :

- La période d'échantillonnage T est de 1 s. C'est un paramètre qui nous est imposé par la nature du signal 1 PPS. On a donc $T = 1 \text{ s}$
- Un retard L nul. En effet, le système est échantillonné toutes les secondes et le système réagit beaucoup plus rapidement qu'une seconde. On considère donc $L = 0 \text{ s}$.
- Une pente $a = 42.67 \text{ Hz/s}$.

Étant donné que le retard est nul et qu'il n'y a pas de dépassement, nous choisissons de réaliser un régulateur PI, ce qui doit normalement suffire pour bien réguler le système. Avec la méthode de Takahashi, on obtient alors :

$$K_p \approx 0.029$$

$$K_i \approx 0.025$$

9.4.2 Implémentation du régulateur

Pour implémenter facilement le régulateur PI, nous utilisons la formule suivante :

$$\text{cmd}_{n+1} = K_p(\varepsilon_{n+1} - \varepsilon_n) + K_i\varepsilon_{n+1} + \text{cmd}_n$$

Avec cmd la commande et ε l'erreur telle que $\varepsilon = \text{consigne} - \text{fréquence mesurée}$.

En effet, cette formule permet de ne pas avoir besoin de stocker le résultat d'une intégrale ou de la somme des erreurs qui pourrait éventuellement déborder de la capacité de la mémoire d'une variable. Cela se fait naturellement en prenant en compte la commande à l'état précédent pour calculer la prochaine commande.

Utilisation de nombres entiers Étant donné que nous sommes sur un microcontrôleur, nous souhaitons éviter d'utiliser des nombres flottants et donc utiliser exclusivement des entiers. Nous multiplions donc les coefficients K_p et K_i par 1000, ce qui nous donne $K_p = 29$ et $K_i = 25$. Il faut ensuite simplement diviser la commande par 1000 lors de son application pour contrebalancer l'effet de la multiplication par 1000 des coefficients.

Point de fonctionnement et ajustement de la commande Notons que le rapport cyclique appliqué est $512 + \text{cmd}$ afin de se placer autour du point de fonctionnement choisi qui est de 512 sur 1023. De plus, afin de rester dans ce domaine, nous limitons la commande cmd à ± 50 (après la division par 1000 donc ± 50000 dans la véritable variable). Lors des premiers tests, nous avons remarqué que le système fonctionnait mal car la commande était trop faible (1 ou 2). Nous avons donc divisé la commande lors de l'application par 100 ou lieu de 1000 (ce qui fait une commande multipliée par 10), ce qui donne des performances satisfaisantes.

Retrait des fonctions inutiles De plus, nous avons vu que le nombre de cycles réalisés par le système d'input capture est constant et égal à 244, ce qui fait que nous n'utilisons pas cette information. Nous décidons alors de retirer la routine qui intervient lors du débordement du timer 1 et qui permet de compter le nombre de cycle réalisé. Cela permet de réduire la taille du code et d'éviter un grand nombre d'interruptions inutiles, ce qui ralentit le code.

Sélection de la consigne À noter que cette commande n'est calculée et appliquée qu'une fois par seconde, ce qui correspond à notre période d'échantillonnage. La valeur de consigne est prise comme étant la fréquence mesurée 5 s après le démarrage du système afin de laisser le temps à la fréquence de se stabiliser avec un rapport cyclique du PWM de 50 % (512/1023). C'est ensuite cette fréquence qui sera maintenue grâce à la régulation.

Test On voit sur la figure 16 en annexe que le régulateur permet d'avoir un système beaucoup moins sensible aux perturbations. De plus, nous avons remarqué que sans régulation, lorsque le résonateur refroidi, la fréquence tend parfois vers une autre valeur que celle avant le chauffage. C'est un effet que l'on ne retrouve pas avec le régulateur puisqu'une commande est appliquée afin que la fréquence retrouve rapidement sa valeur de consigne (on peut observer la consigne sur le graphique du bas).

Globalement, la fréquence avec l'asservissement reste dans un intervalle de ± 2 Hz, ce qui est très satisfaisant. Lors de l'application d'une perturbation, la fréquence varie un petit peu plus mais reste dans un intervalle de ± 5 Hz contrairement au système non régulé qui varie de quelques dizaines de hertz.

Une vidéo de démonstration du fonctionnement en temps réel du système se trouve à l'adresse suivante : <https://youtu.be/HL4xzfdGhLU>

Nous avons démontré que le système est capable de suivre une consigne et de rejeter des perturbations, ce qui remplit donc l'objectif que nous nous étions fixé. Nous pouvons cependant noter que le système ne peut pas rejeter des perturbations trop grandes car sa commande est limitée à ± 50 afin de rester dans une zone de fonctionnement linéaire.

9.5 Watchdog

Nous souhaitons utiliser un chien de garde (watchdog) pour s'assurer que le système ne reste pas bloqué indéfiniment en cas de problème. Cela n'arrive normalement pas, surtout avec un programme si simple et petit que nous avons, mais cela constitue une sécurité. Le programme test de cette fonctionnalité est « Watchdog.c » sur le dépôt Github, même si on le retrouve également implémenté dans « Asservissement_PI.c ».

Fonctionnement Le chien de garde utilise un circuit résonnant peu précis mais indépendant de l'oscillateur du microcontrôleur. Il suffit de simplement appeler régulièrement une fonction qui indique au chien de garde que le microcontrôleur fonctionne normalement. Si cette fonction n'est pas appelée dans le délai imposé par le chien de garde (on peut choisir une valeur entre quelques millisecondes et quelques secondes), alors le microcontrôleur considère que le système s'est bloqué et il redémarre.

Implémentation Il suffit d'inclure le fichier d'entête « avr/wtd.h », puis d'initialiser le chien de garde avec « wtd_enable() ». L'argument de cette fonction indique le temps que met le chien de garde à se déclencher, par exemple 15 ms. Il suffit ensuite d'appeler la fonction « wtd_reset() » dans la boucle infinie pour indiquer que le programme n'a pas planté.

Test Nous testons ce programme seul, puis directement intégré au programme « Asservissement_PI.c ». La valeur d'un délai d'attente maximale de 15 ms est choisie car c'est la valeur la plus faible disponible et est normalement largement suffisante compte tenu du faible nombre de calculs que nécessite notre programme.

Pour vérifier que le chien de garde fonctionne et fait redémarrer le microcontrôleur, nous ajoutons une partie de code provisoire qui permet de forcer une attente de 100 ms (donc supérieure à 15 ms) lorsque le programme a enregistré 5 valeurs.

Lors de nos tests, on voit que le microcontrôleur dès que 5 mesures ont été réalisées et envoyées par USB. On considère donc que le chien de garde fonctionne pour éviter que le système reste bloqué. Nous pensons évidemment à commenter le code qui permet de forcer l'activation du chien de garde afin que le système fonctionne normalement.

10 Conclusion

Ce projet est pour nous une réussite. En effet, nous avons atteint l'objectif d'asservir la fréquence de l'oscillateur d'un microcontrôleur comprenant un résonateur à quartz à l'aide d'un signal 1 PPS.

Lors de la réalisation de ce projet, nous avons appris beaucoup de choses :

- Conception du schéma électrique (KiCAD)
- Simulation du circuit analogique (KiCAD)
- Choix des composants
- Routage du PCB (KiCAD)
- Estimation du coût
- Soudage de composants montés en surface
- Programmation du microcontrôleur (PWM, ICP, Watchdog)
- Synthèse et implémentation d'une loi de commande (Takahashi)
- Visualisation des données (MATLAB)
- Conception d'une boîte pour le PCB (FreeCAD)
- Outils de gestion de projet (Git/Github)

De plus, cela nous a permis de réaliser un travail en groupe, avec les avantages et contraintes que cela implique. Globalement, nous n'avons pas eu de problème de relations humaines mais nous avons remarqué que nous étions parfois trop par rapport à la quantité de travail à réaliser. Peut-être qu'une meilleure gestion des ressources humaine est à envisager, mais nous sommes plutôt satisfaits du travail réalisé et de l'ambiance dans laquelle cela s'est fait, surtout pour une première fois. Cela nous a également montré qu'être beaucoup à travailler sur un projet ne le fait pas forcément avancer plus vite.

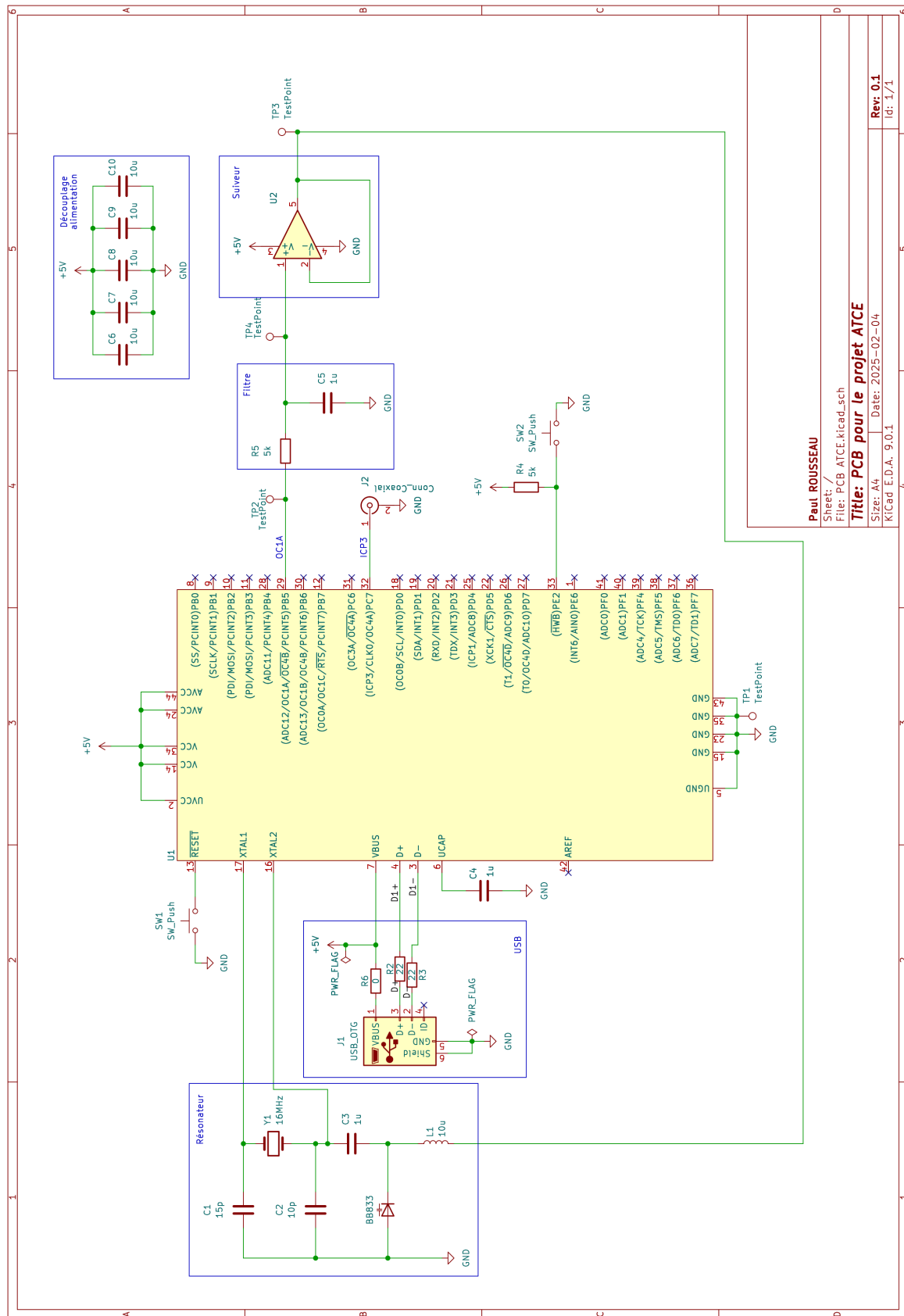
Enfin, nous n'avons pas rencontré de grandes difficultés lors de la réalisation de ce projet hormis la soudure du port USB. En effet, la partie qui aurait pu ralentir le projet est la programmation, mais les programmes ont presque tous fonctionné dès le premier essai. Cela nous a permis de prendre de l'avance et de pouvoir perfectionner notre projet avec l'implémentation d'un chien de garde ou encore de la réalisation d'une boîte en impression 3D. Cela vient aussi du fait que nos enseignants nous ont aidé dès que nécessaire, et nous nous a évité de perdre du temps. De plus, l'utilisation de Git et Github nous ont permis de travailler ensemble efficacement.

Nous profitons de ce compte rendu pour remercier chaleureusement nos enseignants, M. FRIEDT et M. CARRY qui nous ont permis de réaliser ce projet dans de bonnes conditions. Cela nous a permis d'apprendre beaucoup de choses et nous a fait passer une agréable semaine.

11 Bibliographie

- [1] Documentation de l'ATMEGA32U4 :
https://ww1.microchip.com/downloads/en/devicedoc/atmel-7766-8-bit-avr-atmega16u4-32u4_datasheet.pdf
- [2] Fichier BOM format .xlsx à télécharger : https://docs.google.com/spreadsheets/d/1TcTHkYDAkMV4G_8QukV0m2C2ber39rysX6Lb-W6eyqc/edit?usp=sharing
- [3] Modèle de circuit Butterworth-Van Dyke :
<https://demonstrations.wolfram.com/ButterworthVanDykeCircuit>
- [4] Technique de soudure de composants CMS :
<https://youtu.be/us5zLlp1aV0?si=Flcb0uajIgZAHLWc>
- [5] USB OTG broche ID et câble :
<https://www.astuces-pratiques.fr/informatique/usb-otg-broche-id-et-cable>
- [6] Automatique linéaire échantillonnée, Gonzalo Cabodevila (méthode de Takahashi) :
http://jmfriedt.free.fr/Gonzalo_cours1A.pdf

12 Annexes



Reference	Value	Datasheet	Footprint	Qty	Prix/U
B8833	~	https://www.farnell.com/datasheets/1835977.pdf	Diode_SMD:D_0805_2012Metric_Pad1.15x1.40mm_HandSolder	1	0.172€
C1	15p	https://www.yageo.com/en/Chart/Download/pdf/AC0805-IRNPO8N150	Capacitor_SMD:C_0805_2012Metric_Pad1.18x1.45mm_HandSolder	1	0.074€
C2	10p	https://www.yageo.com/en/Chart/Download/pdf/AC0805-IRNPO8N100	Capacitor_SMD:C_0805_2012Metric_Pad1.18x1.45mm_HandSolder	1	0.067€
C3,C4	1u	https://4donline.lhs.com/images/vipMaster/C/IC/YAGO/YAGO-S-A0017891388-1.pdf?hkey=6D3A	Capacitor_SMD:C_0805_2012Metric_Pad1.18x1.45mm_HandSolder	2	0.098€
C5,C6,C7,C8,C9,C10	10u	https://search.kinet.com/component-documentation/download/specsheet/C1206C106J4RACTU	Capacitor_SMD:C_1206_3216Metric_Pad1.33x1.80mm_HandSolder	6	1.230€
J1	USB_OTG	https://www.farnell.com/datasheets/2697485.pdf	Connector_USB:USB_Mini-B_Wuerth_65100516121_Horizontal	1	1.000€
J2	Conn_Coaxial	https://4donline.lhs.com/images/vipMaster/C/IC/AMPH/AMPHS06465-1.pdf?hkey=6D3A4C79FDBF58556	Connector_Coaxial:SMA_Molex_73251-2200_Horizontal	1	6.940€
L1	10u	http://www.farnell.com/datasheets/3006832.pdf	Inductor_SMD:L_0805_2012Metric_Pad1.05x1.20mm_HandSolder	1	0.214€
R1,R6	0	https://4donline.lhs.com/images/vipMaster/C/IC/SCMP/SCMP-S00166-1.pdf?hkey=6D3A4C79FDBF58556	Resistor_SMD:R_0805_2012Metric_Pad1.20x1.40mm_HandSolder	2	0.015€
R2,R3	22	https://www.yageo.com/upload/media/product/app/datasheet/rchip/pvru-rc_group_51_rohs_1.pdf	Resistor_SMD:R_0805_2012Metric_Pad1.20x1.40mm_HandSolder	2	0.015€
R4,R5	5k	https://www.farnell.com/datasheets/3937397.pdf	Resistor_SMD:R_0805_2012Metric_Pad1.20x1.40mm_HandSolder	2	0.015€
SW1,SW2	SW_Push	https://www.farnell.com/datasheets/3148632.pdf	Button_Switch_SMD:SW_Tactile_SPST_NO_Straight_CK_PTS636Sx25SMTRLFS	2	0.937€
TP1,TP2,TP3,TP4	TestPoint	~	TestPoint:TestPoint_Pad_D1.5mm	4	
U1	~	https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7766-8-bit-AVR-ATmega16U4-32U4_Datasheet.pdf	Package_QFP:TQFP-44_10x10mm_P0.8mm	1	5.250€
U2	~	https://www.farnell.com/datasheets/630314.pdf	Package_SO:SOIC-8-N7_3.9x4.9mm_P1.27mm	1	0.353€
Y1	16MHz	https://www.farnell.com/datasheets/3153929.pdf	Crystal:Crystal_SMD_Abracon_ABM3-2Pin_5.0x3.2mm_HandSoldering	1	0.258€

FIGURE 2 – Fichier .csv généré depuis la BOM du schéma

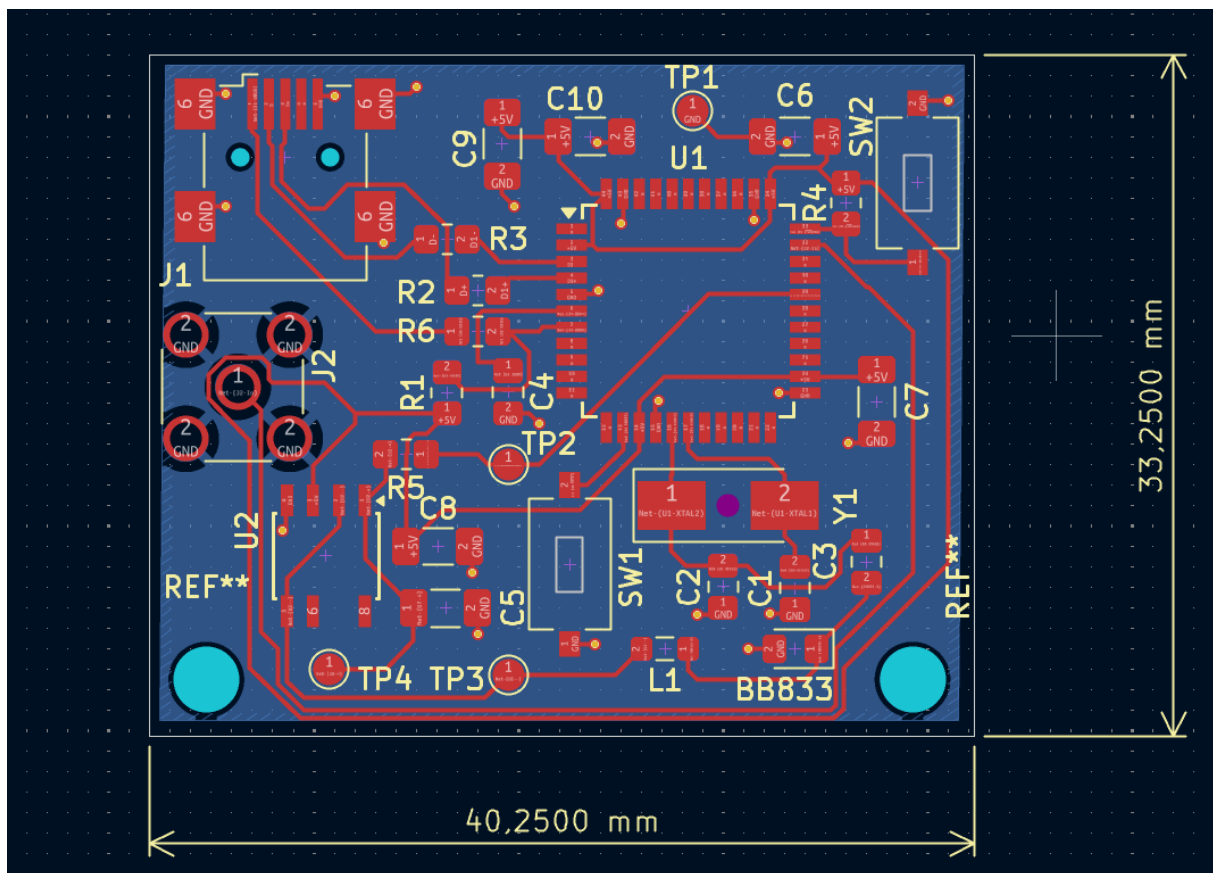


FIGURE 3 – Résultat de la conception du PCB qui utilise 2 couches.

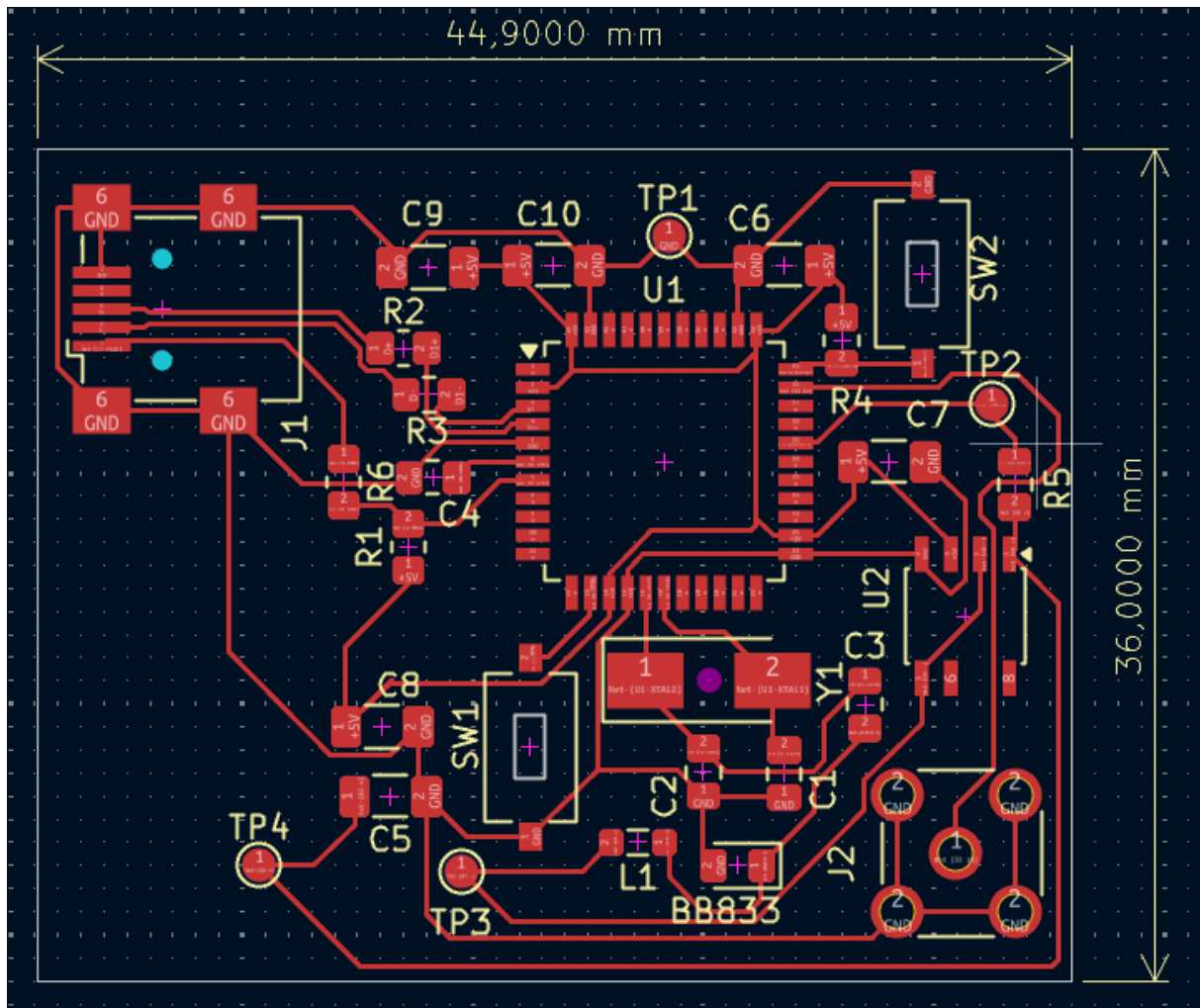


FIGURE 4 – Résultat de la conception du PCB qui utilise une couche.

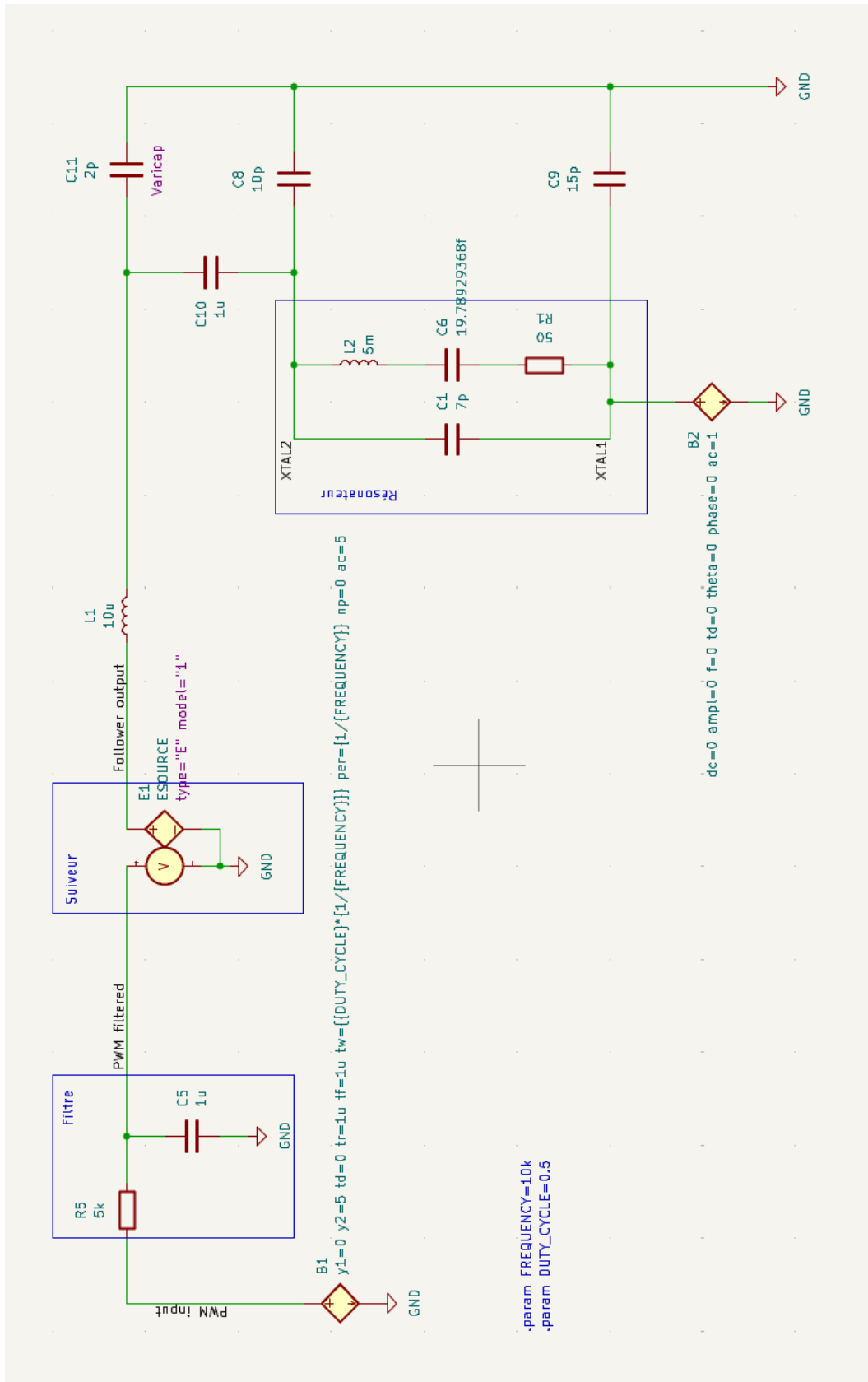


FIGURE 5 – Schéma permettant de simuler le filtre RC et l'oscillateur à quartz avec réglage de fréquence par varicap.

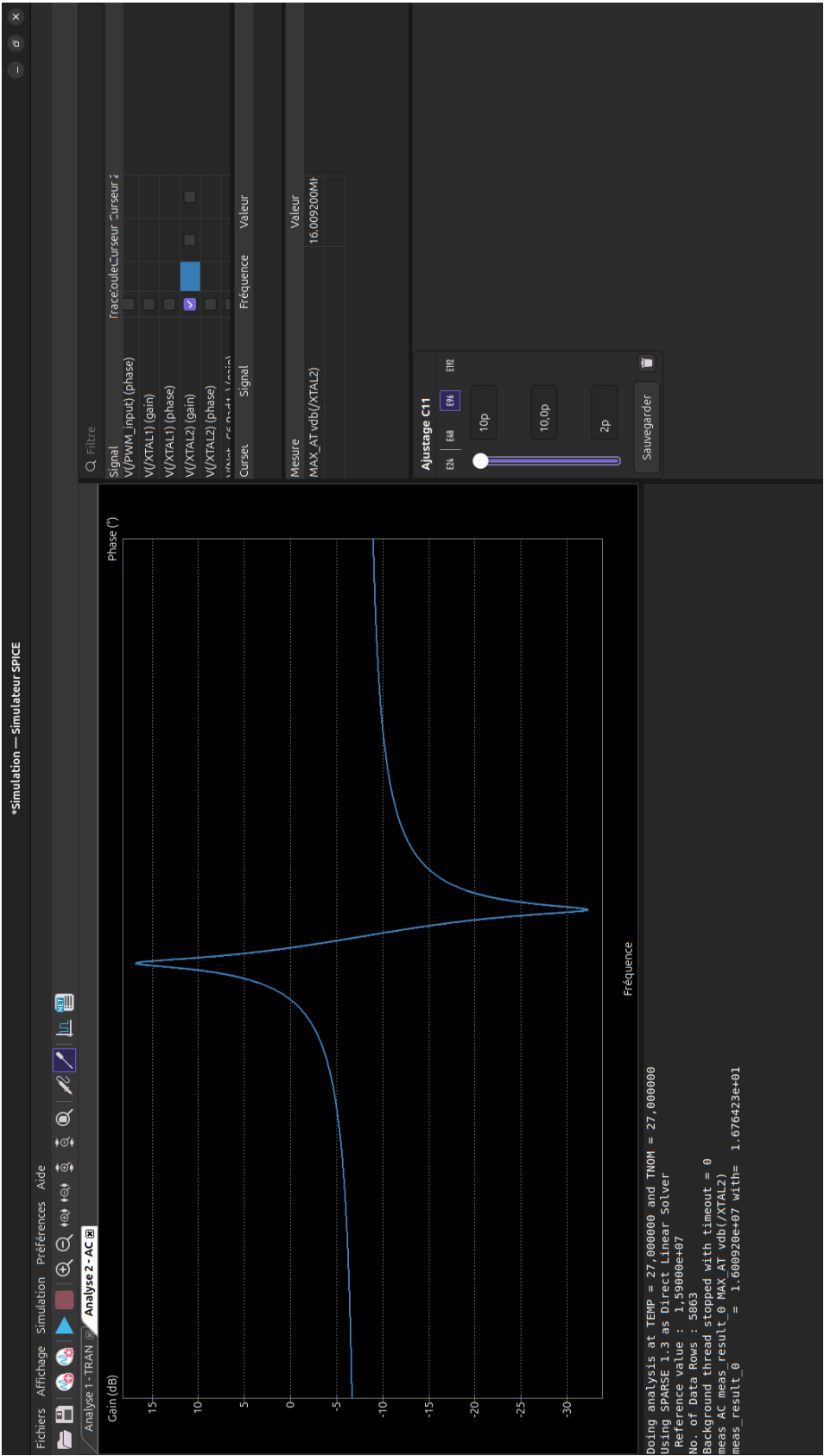


FIGURE 6 – Résultat de l'analyse AC de l'oscillateur à quartz dans KiCad/ngspice.



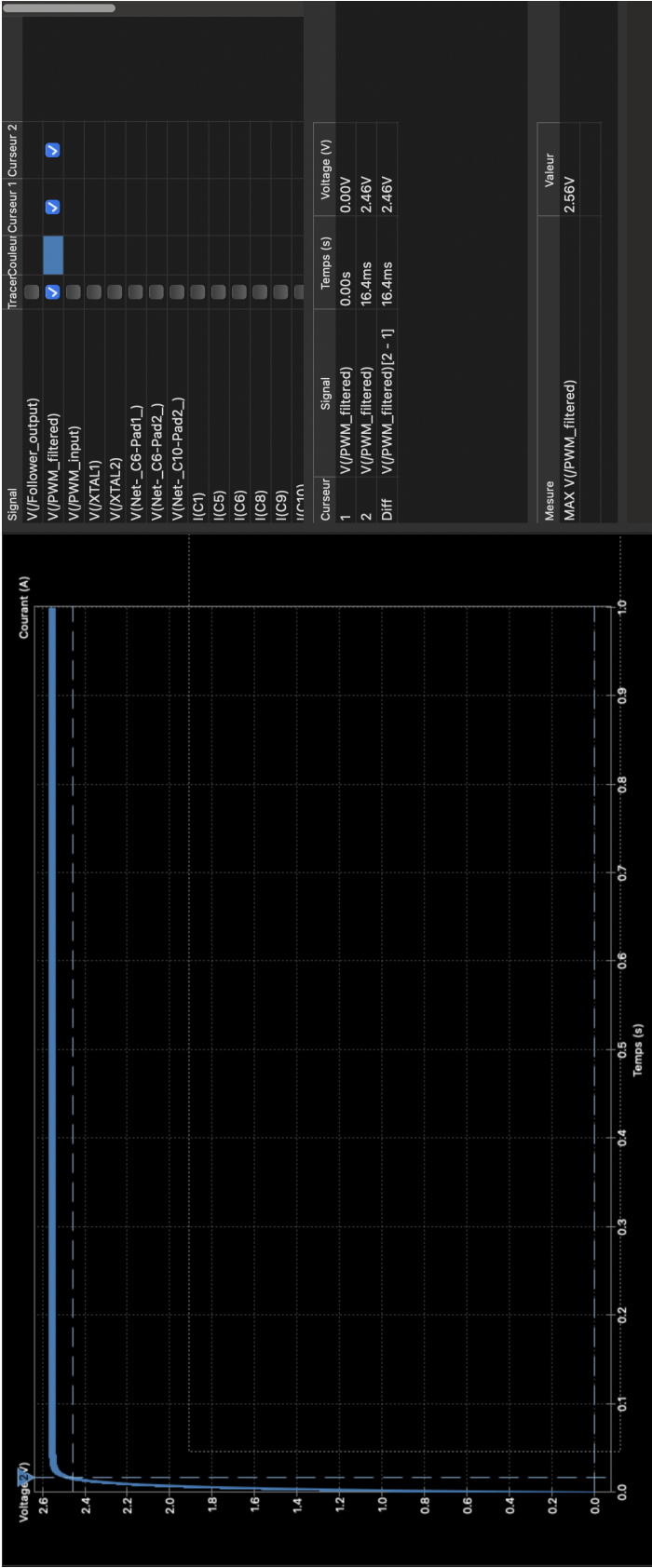


FIGURE 8 – Simulation du filtre RC et visualisation du temps de réponse pour $R = 5,1\text{ k}\Omega$ et $C = 1\text{ }\mu\text{F}$.

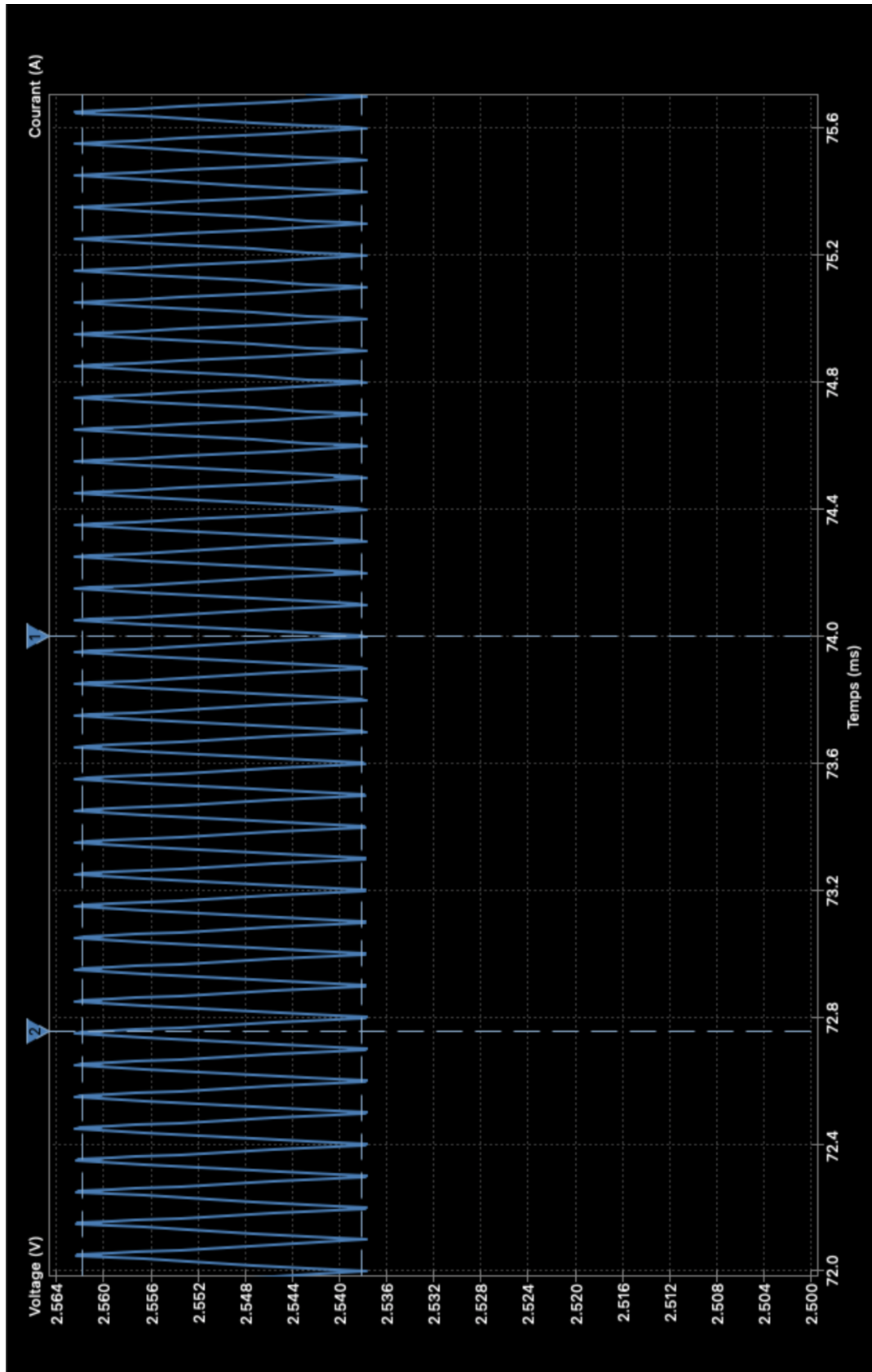


FIGURE 9 – Simulation du filtre RC et visualisation de l'évolution de la tension finale pour $R = 5,1 \text{ k}\Omega$ et $C = 1 \mu\text{F}$.

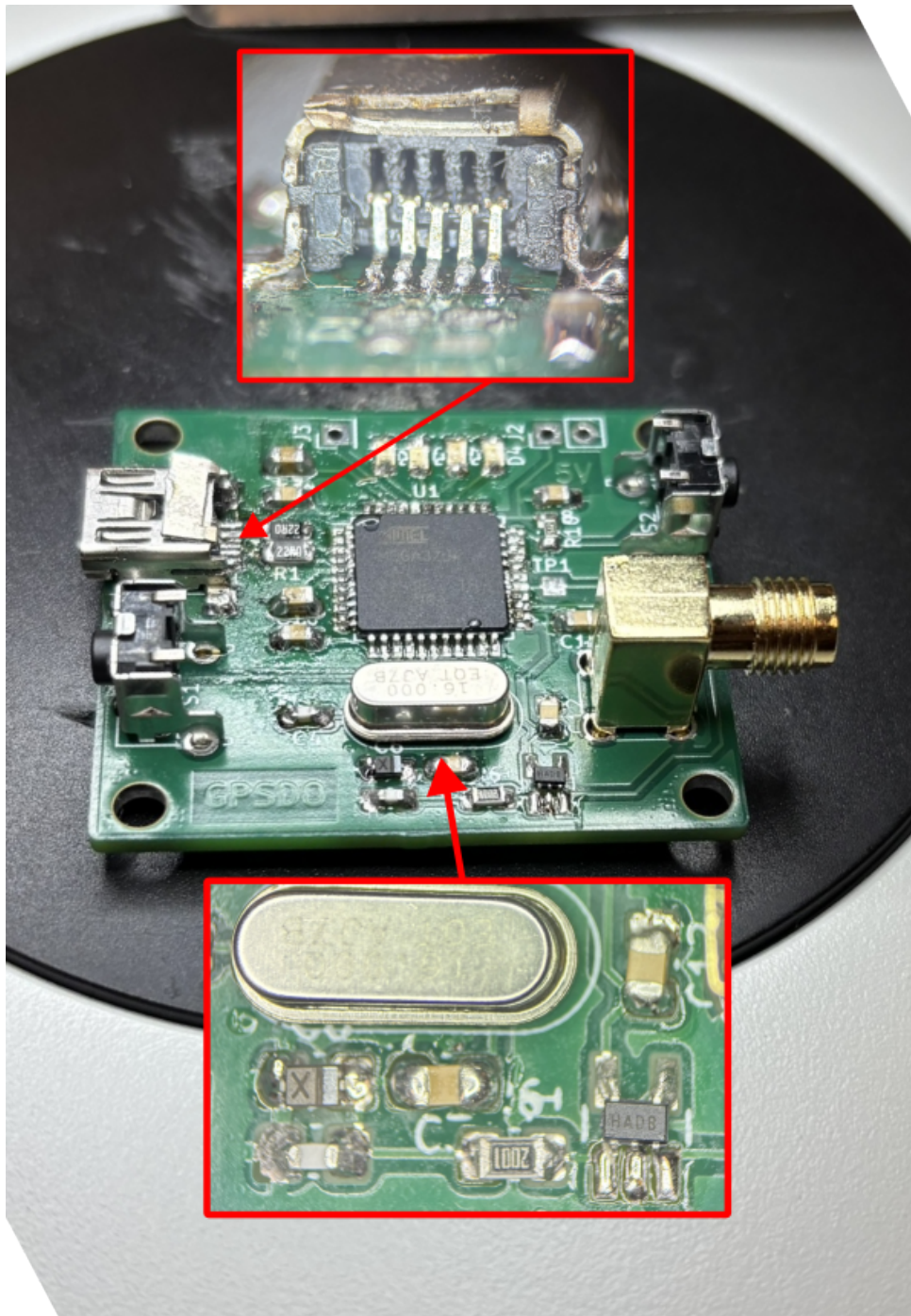


FIGURE 10 – Images du PCB soudé par le groupe, avec un zoom sur le connecteur USB ainsi que l'ensemble qui entoure le quartz.

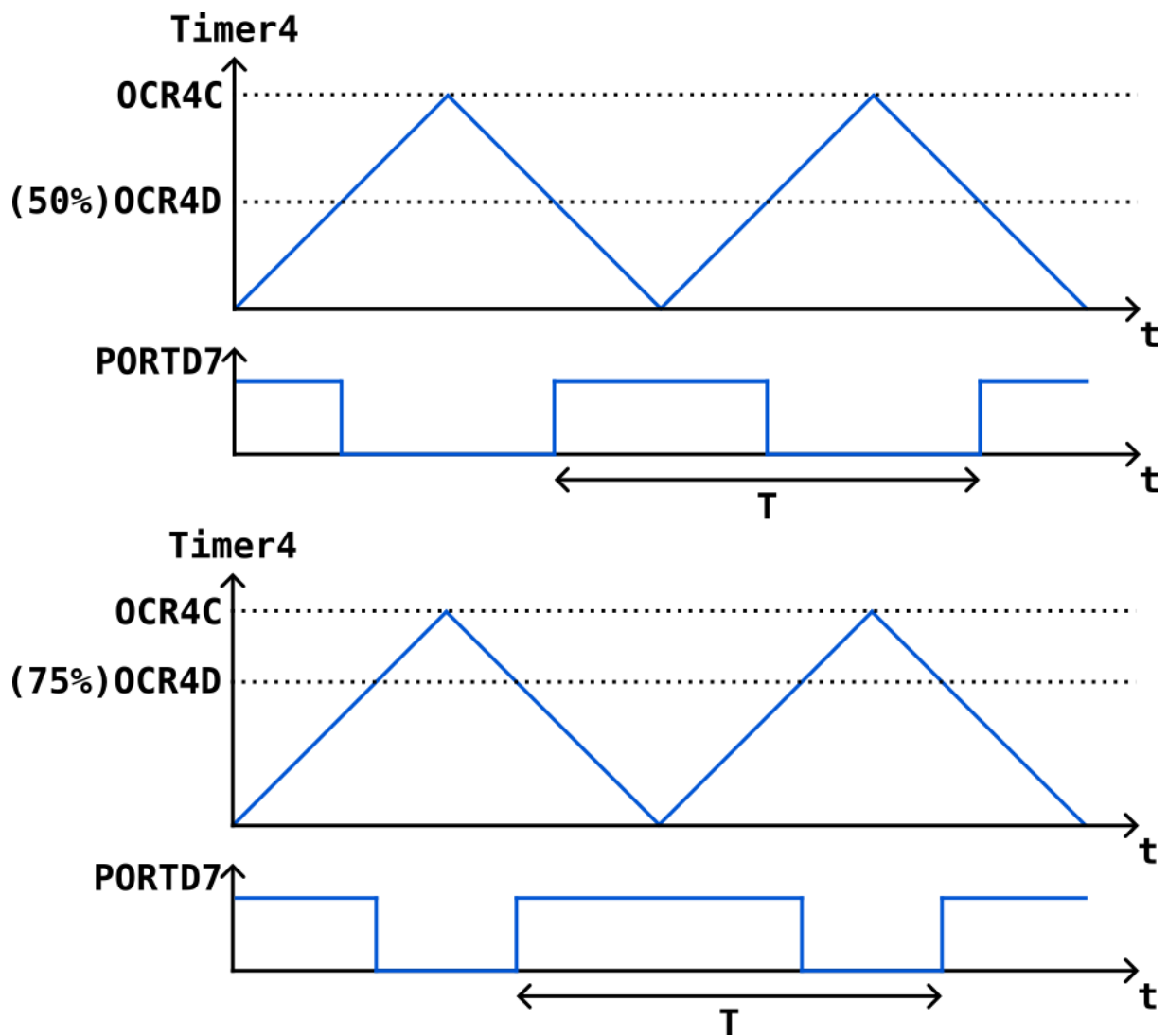


FIGURE 11 – Principe de fonctionnement du PWM et contrôle du rapport cyclique (duty cycle).

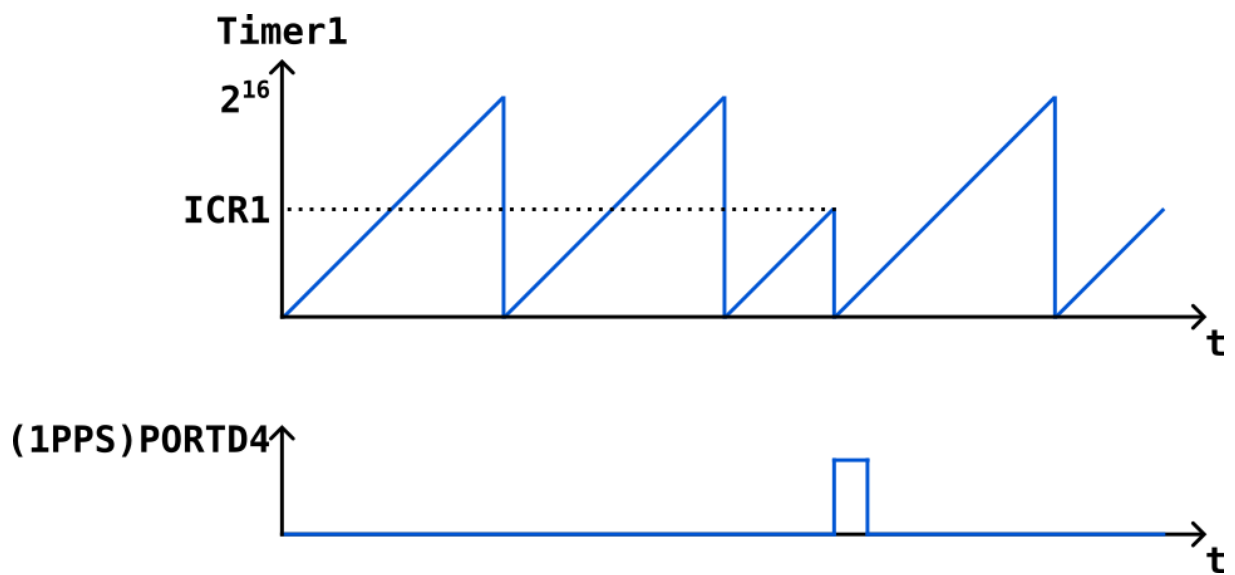


FIGURE 12 – Principe de fonctionnement de l'input capture (ICP) pour mesurer la fréquence de l'oscillateur.

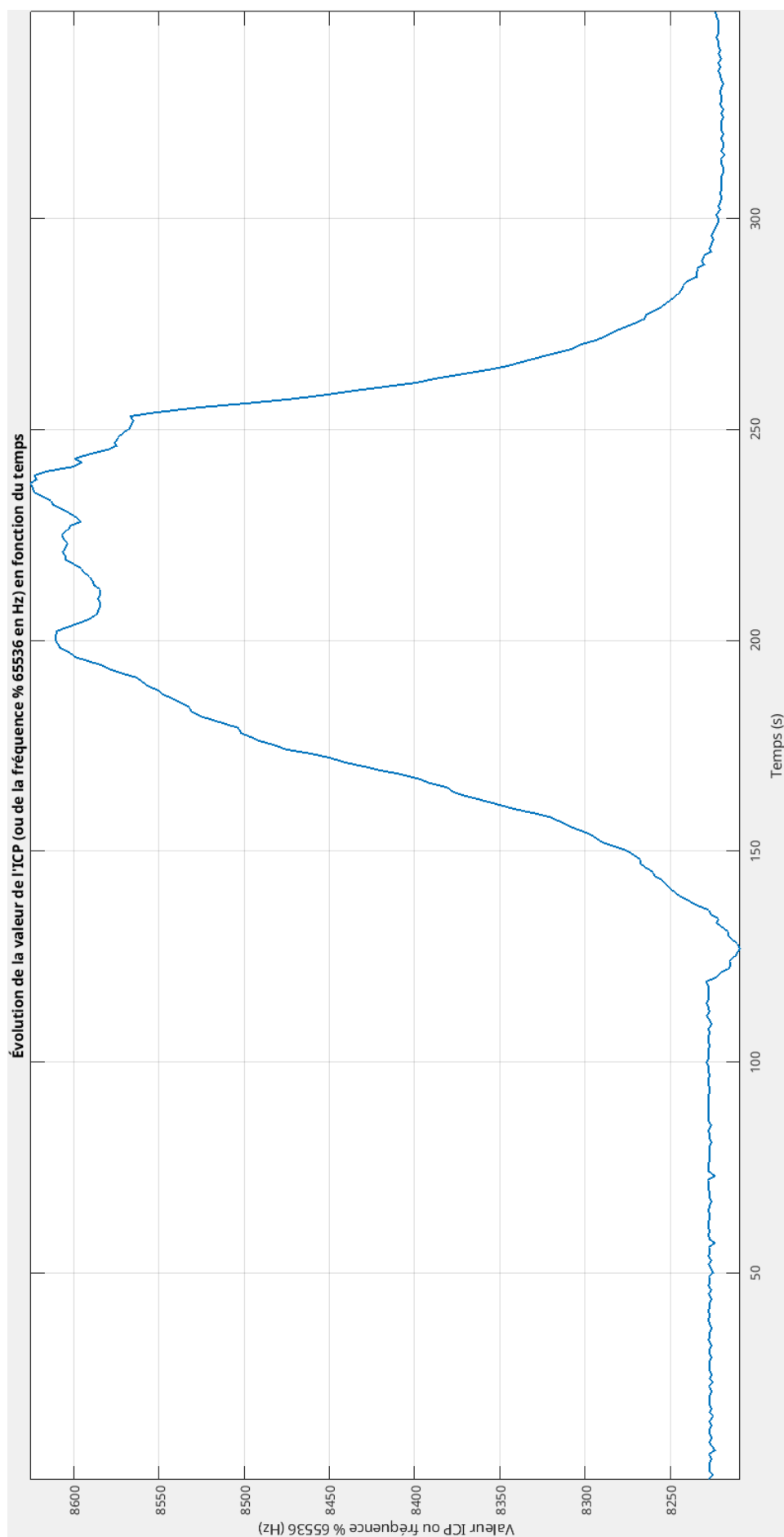


FIGURE 13 – Graphique représentant l'évolution de la valeur de mesure par l'input capture (ICP) en fonction du temps. L'ICP peut également être interprétée comme la fréquence de l'oscillateur modulo 2^{16} en Hz.

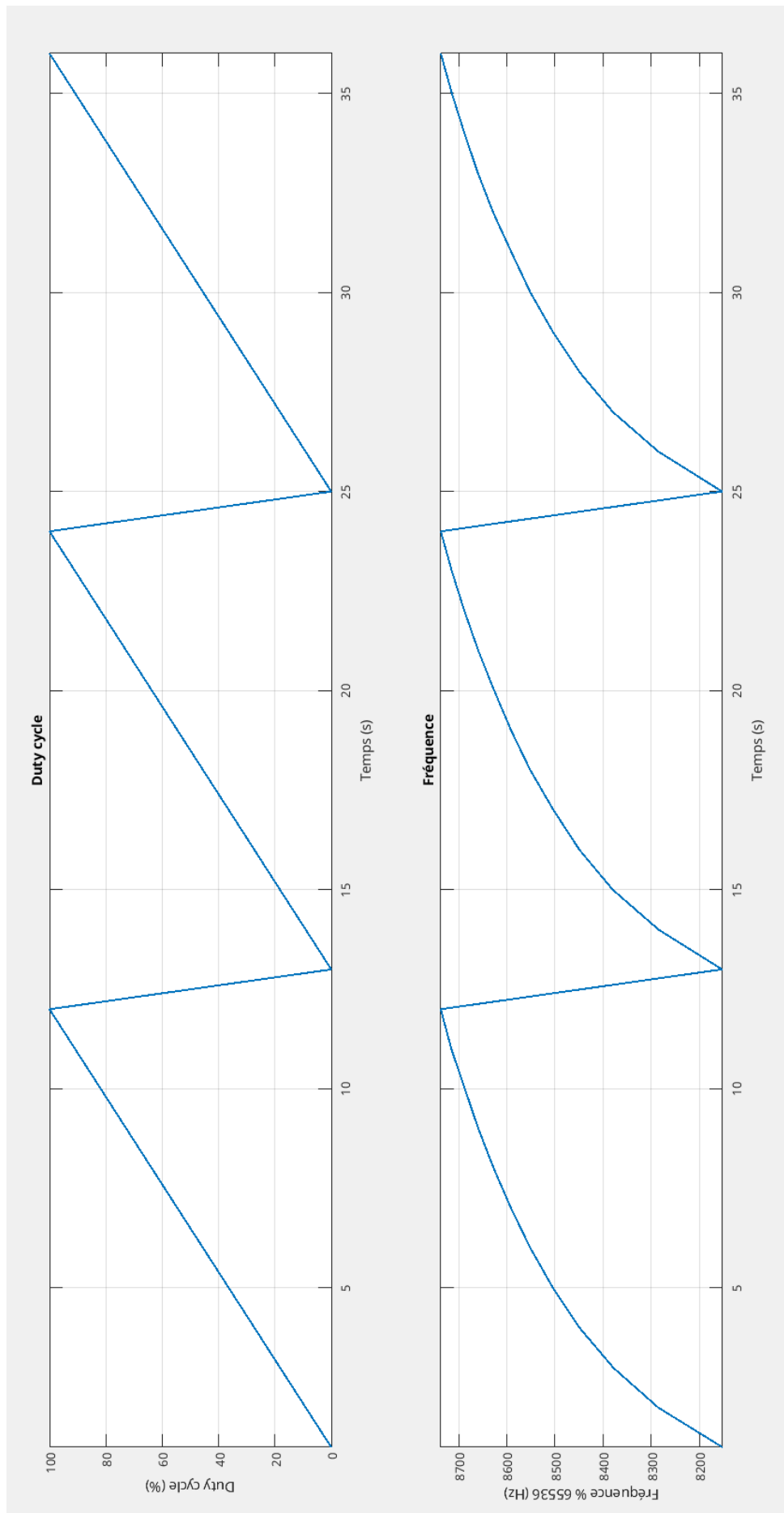


FIGURE 14 – Graphique représentant la fréquence mesurée de l'oscillateur modulo 65536 (2^{16}) en fonction du rapport cyclique du signal PWM, et donc de la tension de polarisation de la Varicap.

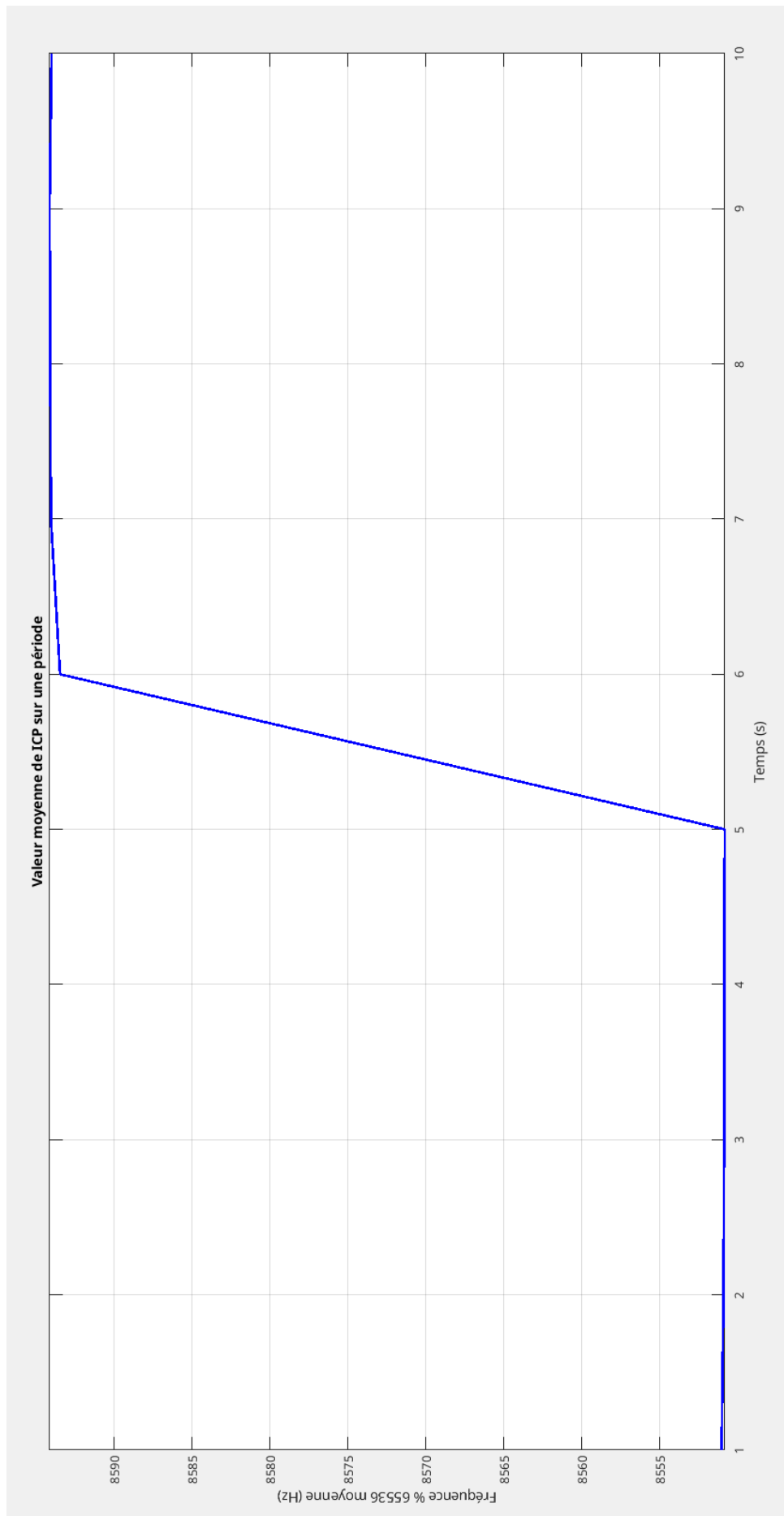


FIGURE 15 – Réponse indicielle moyenne d'un échelon de rapport cyclique passant de 462 à 562 sur 1023.

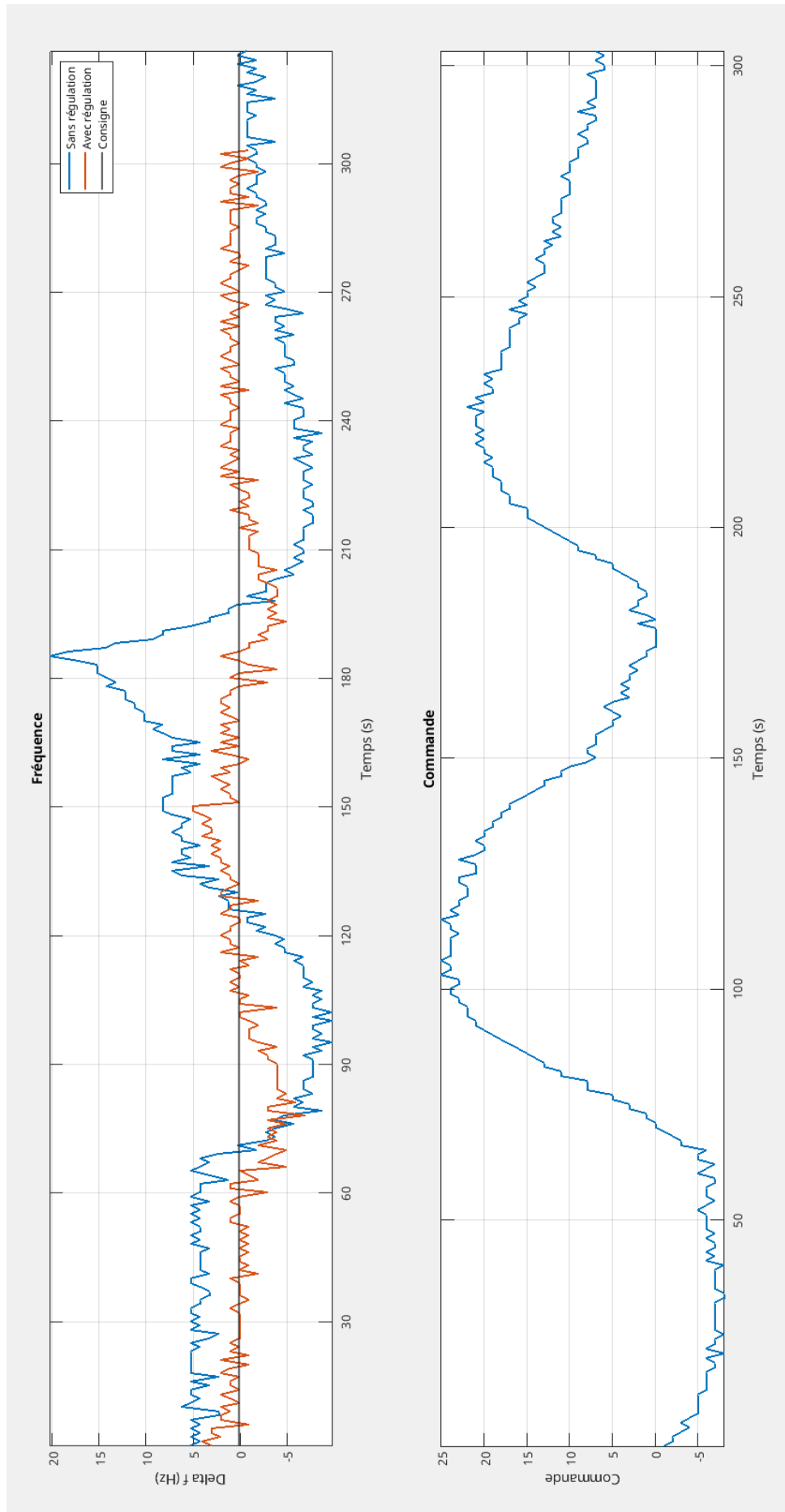


FIGURE 16 – Graphique comparant l'évolution de la fréquence de l'oscillateur avec et sans régulation (graphique du haut). On peut voir la commande sur le graphique du bas qui permet de compenser l'effet de la température.

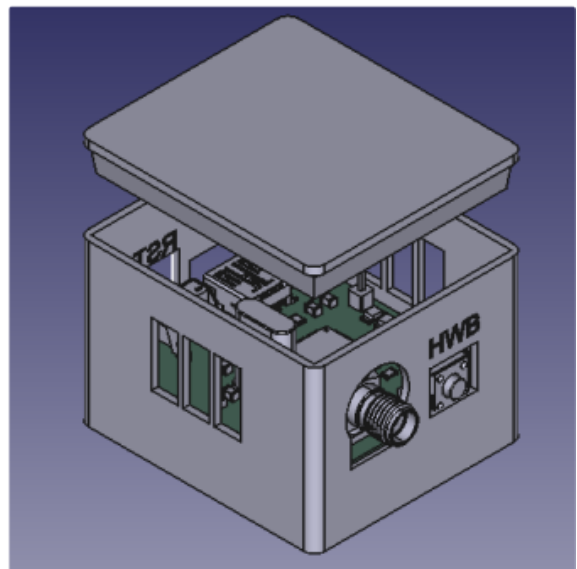
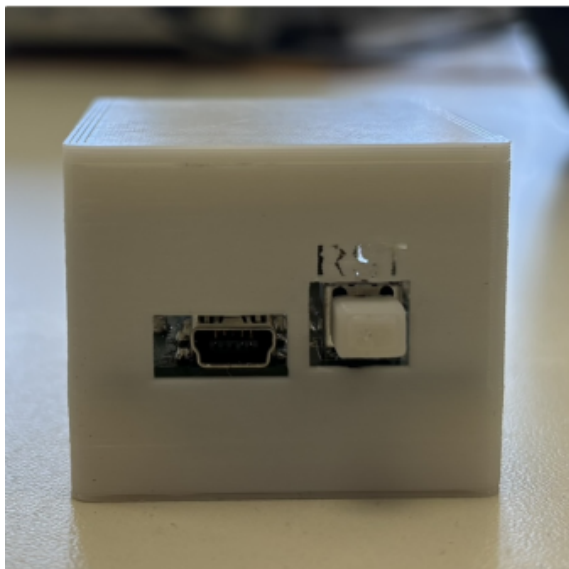
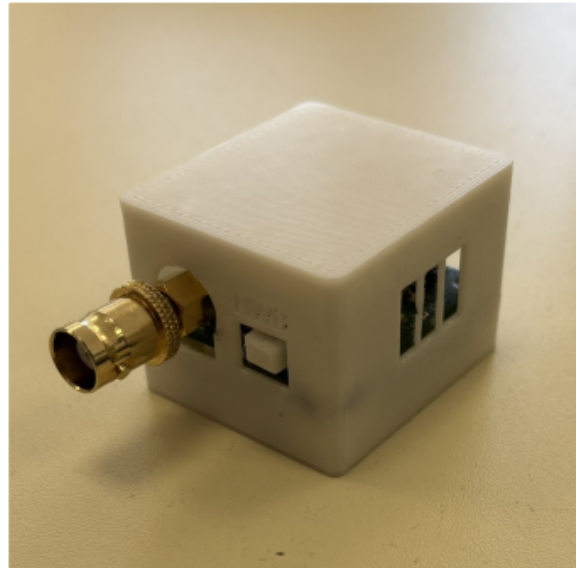


FIGURE 17 – Images du boîtier contenant le PCB.